

Algoritmos de Alginvesting

X4 — Backtester · X5 — Cerebro macro

Pseudocódigos comentados, de lo global a lo particular

Proyecto Alginvesting

August 26, 2026

Este documento describe con precisión operativa los dos módulos más grandes y menos visibles del sistema: `X4_backtester.py` (simulador histórico de la estrategia) y `X5_macro_brain.py` (modelo que aprende qué parámetros conviene usar en cada contexto). Cada sección parte de la idea global y baja al detalle línea a línea. Los conceptos técnicos se definen en una nota al pie la primera vez que aparecen, para no obligar a saltar al glosario.

Índice

1	Panorama general	4
1.1	Qué problema resuelve el sistema completo	4
1.2	Por qué X4 y X5 se explican juntos	4
1.3	Cómo leer este documento	4
1.4	Vocabulario del dominio	5
1.4.1	La mecánica de una operación, con números	5
2	X4 — Backtester	7
2.1	Propósito y contrato	7
2.1.1	Qué hace, en una frase	7
2.1.2	Qué garantiza: ausencia de información del futuro	7
2.1.3	Qué no simula: los supuestos que hay que tener presentes	7
2.1.4	Los dos modos de ejecución	8
2.1.5	Entradas y salidas	8
2.2	Arquitectura interna	8
2.2.1	El estado: una sola estructura que lo contiene todo	8
2.2.2	Checkpoint: parar y seguir	9
2.2.3	Carga de datos	9
2.3	El algoritmo global	9
2.4	Los pasos A–F: la lógica de trading de una vela	11
2.4.1	Paso A — cancelar órdenes en espera huérfanas	11
2.4.2	Paso B — ejecutar las órdenes que el precio alcanzó	12
2.4.3	Paso C — el stop protector móvil	13
2.4.4	Paso D — cierre por pérdida máxima	14
2.4.5	Paso E — cierre por stop tocado	14
2.4.6	Paso F — crear órdenes en espera	14
2.4.7	Por qué este orden y no otro	15
2.5	Simulación intra-vela	16
2.5.1	El problema	16
2.5.2	La solución adoptada	16
2.5.3	Limitaciones honestas de este enfoque	17
2.6	Contabilidad de la cuenta	17
2.6.1	Las cuatro magnitudes	17

2.6.2	Quiebra de la cuenta	17
2.6.3	Los dos archivos de patrimonio	18
2.7	Recálculo de soportes dentro del backtest	18
2.7.1	Delegación en X0	18
2.7.2	Paralelismo por activo	18
2.7.3	El caché de backtesting y el arranque en caliente	18
2.7.4	Las posiciones abiertas son anclas	19
2.8	Persistencia de operaciones y eventos	19
2.8.1	Los dos registros	19
2.8.2	Identificadores reconstruibles	19
2.8.3	Escritura segura	19
2.9	Modo X5: X4 al servicio de la recolección	19
2.9.1	Uno: los parámetros cambian durante la corrida	19
2.9.2	Dos: cada operación cerrada se convierte en una fila de entrenamiento	20
2.9.3	Tres: filas periódicas	21
2.9.4	Cuatro: aislamiento por activo	21
2.10	Parámetros de X4 y su efecto	21
3	X5 — Cerebro macro	22
3.1	Propósito	22
3.1.1	El problema: una configuración fija para un mercado que no lo es	22
3.1.2	La idea: un modelo sustituto	22
3.1.3	El ciclo cerrado	22
3.1.4	Las dos fases de despliegue	22
3.2	Interfaz de línea de comandos	23
3.3	El store: el conjunto de datos	23
3.3.1	Un archivo por activo	23
3.3.2	Los dos tipos de fila	23
3.3.3	Los bloques de columnas	23
3.3.4	Los tres objetivos	24
3.3.5	Escritura desde X4	24
3.4	Recolección	24
3.4.1	El orquestador	24
3.4.2	El worker de un activo	25
3.4.3	El recorrido guiado	25
3.4.4	Reiniciar un activo	26
3.5	Entrenamiento	26
3.5.1	Qué modelo, según cuántos datos haya	26
3.5.2	Preparación del vector de entrada	26
3.5.3	LightGBM: tres modelos independientes	27
3.5.4	FT-Transformer: un tronco y tres cabezas	28
3.5.5	Métricas y su historial	29
3.6	Inferencia: de un modelo entrenado a una recomendación	29
3.6.1	El contexto	29
3.6.2	Optimización sobre LightGBM: búsqueda bayesiana	30
3.6.3	Optimización sobre el FT-Transformer: ascenso por gradiente	30
3.6.4	El airbag: una regla que el modelo no puede sobrescribir	31
3.6.5	La salida: <code>active_parameters.json</code>	31
3.7	Pseudocódigo consolidado de X5	32
3.8	Parámetros de X5 y su efecto	32

4	Interacción X4 ↔ X5	33
4.1	Quién llama a quién	33
4.2	Los tres relojes	33
4.3	Una recolección completa, paso a paso	34
4.4	Aislamiento: qué separa a qué	34
4.5	Supuestos y riesgos conocidos	35
4.6	Cómo saber si el bucle está funcionando	35
5	Anexos	36
5.1	Glosario técnico	36
5.2	Mapa de archivos generados	36
5.3	Índice de funciones	37
5.3.1	X4_backtester.py	37
5.3.2	X5_macro_brain.py	38

1 Panorama general

1.1 Qué problema resuelve el sistema completo

Alinvesting compra en niveles de precio que el mercado ya respetó antes. La idea de fondo es simple: si un activo bajó muchas veces hasta cierto precio y desde ahí rebotó, ese precio concentra órdenes de compra reales; volver a comprar ahí tiene, en promedio, mejor relación entre ganancia y riesgo que comprar en cualquier punto al azar. El sistema mantiene decenas o cientos de esos niveles activos por activo, coloca órdenes de compra pendientes en ellos y las administra con un mecanismo de protección de ganancias.

Ese objetivo se reparte en seis scripts. La Tabla 1 los sitúa. Este documento cubre solo los dos últimos, pero es imposible entenderlos sin saber qué le entrega cada uno de los anteriores.

Script	Rol	Qué produce
X0	Datos y soportes	Descarga velas ¹ horarias y calcula, por optimización, los N soportes óptimos de cada activo.
X1	Trading en vivo	Lee los soportes, coloca y gestiona órdenes reales en MetaTrader 5.
X2	Score fundamental	Un número en $[0, 1]$ por activo que resume su salud fundamental.
X3	Features técnicas	Indicadores técnicos por vela (medias móviles, RSI, ATR, etc.).
X4	Backtester	Simula X0+X1 sobre el pasado; produce trades, eventos y curva de capital.
X5	Cerebro macro	Aprende qué parámetros usar según el contexto y los escribe en un archivo.

Tabla 1: Los seis scripts del sistema. Este documento cubre X4 y X5.

1.2 Por qué X4 y X5 se explican juntos

X4 y X5 se escribieron como dos programas separados, pero funcionan como uno solo. La relación es circular y conviene fijarla desde ya:

- **X5 necesita datos que no existen.** Para aprender qué parámetros funcionan mejor en cada contexto hacen falta miles de operaciones cerradas, cada una etiquetada con el contexto de mercado y los parámetros vigentes cuando se abrió. En vivo eso tomaría años. X5 los fabrica corriendo X4 sobre el histórico.
- **X4 necesita saber con qué parámetros simular.** Si siempre simulara con los mismos, el conjunto de datos resultante no tendría variedad y ningún modelo podría aprender de él. Por eso, en modo `--x5`, X4 cambia de parámetros cada pocos días simulados: a veces al azar, a veces preguntándole al modelo que X5 ya entrenó.

En una frase: **X4 es el laboratorio y X5 es el experimentador.** X5 diseña las condiciones del experimento, X4 lo ejecuta y devuelve los resultados, X5 aprende de ellos y diseña el siguiente. La Sección 4 desarma ese bucle paso a paso, una vez que ambos módulos se hayan explicado por separado.

1.3 Cómo leer este documento

La estructura es deliberadamente descendente. Cada uno de los dos capítulos centrales (Secciones 2 y 3) sigue el mismo orden:

1. **Propósito y contrato:** qué entra, qué sale, qué garantiza y qué no.
2. **Arquitectura:** las piezas internas y cómo se relacionan.

3. **Pseudocódigo global:** el algoritmo completo en una página, sin detalles.
4. **Detalle:** cada pieza del pseudocódigo global, expandida.
5. **Parámetros:** qué controla cada perilla y qué pasa al moverla.

Los pseudocódigos están numerados por sección (Pseudocódigo 2.1, 2.2, ...) y nombran las funciones reales del archivo .py correspondiente, de modo que se pueda saltar del documento al código sin traducción intermedia. Cuando una línea del pseudocódigo simplifica algo relevante, hay una nota al pie o un párrafo inmediatamente después.

1.4 Vocabulario del dominio

Todo el documento usa las siglas de la Tabla 2. Están definidas también en CLAUDE.md, pero se repiten aquí porque el resto del texto las da por sabidas.

Sigla	Significado
Soporte	Nivel de precio en el que se coloca una orden de compra pendiente.
N	Cantidad de soportes activos por activo.
O0	Orden potencial: su soporte todavía está demasiado cerca del precio actual, así que aún no se coloca.
OE	Orden en Espera: orden de compra límite ² ya colocada, esperando que el precio baje hasta ella.
OA	Orden Abierta: la OE se ejecutó; hay una posición viva ganando o perdiendo.
OC	Orden Cerrada: la posición se cerró (por protección de ganancias, por pérdida máxima o por stop).
A	Ganancia flotante en dólares que debe alcanzar una posición para que se active por primera vez su stop protector.
B	Distancia en dólares a la que el stop protector sigue al precio una vez activado.
L	Tamaño efectivo de la posición: lote $\times$ unidades por lote. Convierte diferencias de precio en dólares.
FO	Función Objetivo: número que mide qué tan bueno es un conjunto de N soportes. X_0 la maximiza.
T	Ventana de días de historia usada para calcular soportes.

Tabla 2: Vocabulario mínimo. La progresión natural de una orden es $O0 \rightarrow OE \rightarrow OA \rightarrow OC$.

1.4.1 La mecánica de una operación, con números

Antes de entrar en el código conviene tener en la cabeza una operación completa, porque todo X_4 no es más que repetir esto millones de veces.

Supón NVDA cotizando a \$180. Hay un soporte en \$174. Los parámetros son $A = 6$, $B = 2$, lote = 0,01, unidades por lote = 100, es decir $L = 0,01 \times 100 = 1$ (cada dólar de movimiento del precio vale un dólar en la cuenta).

1. **Creación ($O0 \rightarrow OE$).** La distancia entre el precio y el soporte, medida en dólares, es $(180 - 174) \times L = 6$. Como $6 \geq A = 6$, la orden se coloca. Si el soporte estuviera en \$176, la distancia sería \$4 y la orden quedaría en O0: demasiado cerca, no vale la pena inmovilizar una orden ahí.
2. **Ejecución ($OE \rightarrow OA$).** El precio baja y en algún momento una vela toca \$174. La orden se ejecuta: ahora hay una posición comprada a \$174.
3. **Activación del stop protector (paso C).** El precio sube a \$181. La ganancia flotante es $(181 - 174) \times L = 7 \geq A = 6$, así que se activa por primera vez un stop en $181 - B/L = 181 - 2 = \$179$. Desde este momento, la operación no puede terminar en pérdida.

4. **Seguimiento.** El precio sigue a \$184: el stop se mueve a $184 - 2 = \$182$. Nunca baja; si el precio retrocede a \$183, el stop se queda en \$182.
5. **Cierre (OA $\rightarrow$ OC).** El precio cae a \$182 y toca el stop. La posición se cierra con $(182 - 174) \times L = \$8$ de ganancia. Se registra una fila OC.

El camino alternativo es el malo: el precio nunca sube lo suficiente para activar el stop y en cambio cae hasta que $(174 - \text{precio}) \times L > \text{PERDIDA_MAX}$. Entonces la posición se cierra a pérdida (paso D). Toda la estrategia vive en la tensión entre esos dos finales, y todos los parámetros que X5 optimiza mueven la frontera entre uno y otro.

2 X4 — Backtester

2.1 Propósito y contrato

2.1.1 Qué hace, en una frase

X4 toma el historial de precios ya descargado, se para en la primera vela y avanza hora a hora hasta el presente, comportándose exactamente como se habría comportado el sistema real en cada uno de esos momentos: calcula soportes con la información disponible hasta esa hora, coloca órdenes, las ejecuta cuando el precio las alcanza, mueve stops, cierra posiciones y anota todo. Al final entrega un historial completo de operaciones y una curva de capital.

Esto es *backtesting*³. Su utilidad es doble: permite estimar si la estrategia gana o pierde dinero antes de arriesgar capital real, y —el uso dominante hoy— genera el conjunto de datos de entrenamiento de X5.

2.1.2 Qué garantiza: ausencia de información del futuro

El error clásico de un backtester es dejar entrar, sin darse cuenta, información que en el momento simulado todavía no existía. X4 se protege de eso en tres puntos concretos:

- **Soportes.** Cuando en el instante simulado t hay que recalcular soportes, se llama a la misma función de X0 pasándole `fecha_hora_max = t`. Esa función recorta el CSV a {filas con fecha $\leq t$ } antes de optimizar. Un soporte nunca se apoya en una vela que aún no ocurrió.
- **Features de contexto.** El fotograma de indicadores técnicos (X3) y de score fundamental (X2) que acompaña a cada operación se calcula también con el corte `df.index <= ts`, y para X2 se elige la entrada del historial cuya fecha sea $\leq$ la fecha simulada.
- **Columnas prohibidas en el modelo.** X5 excluye explícitamente del vector de entrada cualquier columna terminada en `_oc` (información del momento del cierre), que por definición no está disponible cuando hay que decidir.

2.1.3 Qué no simula: los supuestos que hay que tener presentes

Un backtester es siempre una simplificación. Los supuestos de X4, ordenados de más a menos relevante:

1. **Sin deslizamiento ni comisiones.** Una orden límite en \$174 se ejecuta a \$174 exactos, salvo el caso de hueco (Sección 2.4.2). No se descuentan comisiones ni costo de financiamiento nocturno. El resultado simulado es, por tanto, un techo optimista.
2. **Liquidez infinita.** Se asume que siempre hay contraparte para el tamaño operado. Con los lotes mínimos que usa el sistema es un supuesto razonable.
3. **Intra-vela sintético.** Dentro de una vela de una hora, el orden real en que el precio visitó su máximo y su mínimo es desconocido. X4 lo reconstruye tomando prestado un tramo real de sesenta minutos y reescalándolo (Sección 2.5). Es una aproximación, y es la mayor fuente de incertidumbre del simulador.
4. **Sin cierre forzado al final.** Cuando se acaban los datos, las posiciones abiertas quedan abiertas: no se liquidan a precio de mercado. El capital final reportado es saldo realizado, no valor de liquidación.

³**Backtesting:** evaluar una estrategia aplicándola sobre datos históricos como si se estuviera operando en ese momento. Su validez depende por completo de no usar información del futuro (ver Sección 2.1.2).

2.1.4 Los dos modos de ejecución

El mismo archivo se usa para dos cosas distintas, y confundirlas es la fuente habitual de desorientación. La Tabla 3 las separa.

	Modo versión	Modo X5
Invocación	<code>python X4_backtester.py</code> <code>--version V1</code>	<code>python X4_backtester.py --x5</code> <code>--activo BTCUSD</code>
Configuración	<code>resources/x4/versionV1/config_V1.py</code>	<code>scripts/config_x5_default.py</code>
Parámetros	Fijos durante toda la corrida	Se regeneran cada pocos días simulados
Pregunta que responde	¿Cuánto habría rendido esta configuración exacta?	¿Cómo rinden <i>muchas</i> configuraciones en <i>muchos</i> contextos?
Salida principal	<code>trades.json</code> , <code>equity_global.csv</code>	<code>resources/x5/{ACTIVO}_store.csv</code>
Activos	Todos los de la versión	Uno solo (aislamiento para paralelizar)

Tabla 3: Los dos modos de X4. El interruptor es el argumento `--x5`.

El modo versión existe para responder preguntas puntuales y reproducibles: cada versión congela su propia copia de la configuración en un archivo aparte, de modo que cambiar `config.py` meses después no altere el resultado de una corrida antigua. El script auxiliar `X4B_crear_version_backtesting.py` crea ese andamiaje (carpeta, plantilla de configuración, registro de la versión en `config.py`) a partir de un nombre.

2.1.5 Entradas y salidas

Dirección	Archivo	Contenido
Entrada	<code>Data/{ACTIVO}.csv</code>	Velas H1 con OHLCV. Obligatorio.
Entrada	<code>Data_minuto/{ACTIVO}.csv</code>	Velas M1. Opcional: sin él, la simulación intra-vela se degrada.
Entrada	<code>resources/x2/x2_history.json</code>	Historial de scores fundamentales (solo modo X5).
Salida	<code>checkpoint.json</code>	Estado completo para reanudar.
Salida	<code>trades.json</code>	Una entrada por operación cerrada.
Salida	<code>events.json</code>	Bitácora de todo: creación, cancelación, cambio de stop, cierre.
Salida	<code>equity_global.csv</code>	Una fila por vela: saldo, patrimonio, margen.
Salida	<code>equity_activos.csv</code>	Una fila por vela y activo: ganancia cerrada, flotante y total.
Salida	<code>conjuntos_N/{V}_{N}_bt.json</code>	Caché de soportes indexado por fecha.
Salida	<code>{ACTIVO}_store.csv</code>	Filas de entrenamiento para X5 (solo modo X5).

Tabla 4: Contrato de archivos de X4. Todas las rutas salen de la configuración activa.

2.2 Arquitectura interna

2.2.1 El estado: una sola estructura que lo contiene todo

X4 no usa clases. Todo el estado de la simulación vive en un único diccionario anidado que se pasa por referencia a cada función. Conocer su forma es conocer el programa:

```
estado = {
    'capital': 3000.0,                # saldo realizado en USD
    'ts_ultimo_procesado': '2026-03-04T11:00:00',
    'ts_ultimo_recalculo': '2026-03-01T09:00:00',
    'stop_out': False,
    'por_activo': {
```

```

'BTCUSD': {
    'soportes': [61200.0, 61845.5, ...],          # lista ordenada de N precios
    'OE': {61200.0: {'lote':0.01, 'ts_creacion':...}, ...},
    'OA': {60310.0: {'lote':0.01, 'sl':60480.0, 'ts_apertura':...,
                    'ganancia_max':..., 'drawdown_max':...,
                    'duracion_velas':..., 'capital_apertura':...}, ...},
    'GC': 184.55,                                # ganancia cerrada acumulada
},
'ETHUSD': {...},
},
}

```

Dos decisiones de diseño se leen directamente ahí:

- **Las órdenes se indexan por precio, no por identificador.** OE y OA son diccionarios cuya clave es el precio de la orden. Esto hace instantáneo preguntar «¿ya hay una orden en este soporte?», que es la consulta más frecuente del bucle, pero impone una restricción: no puede haber dos órdenes en el mismo precio exacto para el mismo activo. En la práctica no ocurre, porque los soportes son distintos entre sí por construcción.
- **El estado es serializable.** Todo lo que hay ahí adentro son números, textos y diccionarios: se puede volcar a JSON y recuperar tal cual. De eso vive el mecanismo de reanudación.

2.2.2 Checkpoint: parar y seguir

Una pasada completa sobre varios años de velas horarias tarda horas. Cada 24 velas procesadas — y siempre antes de un evento importante— X4 vuelca `estado` a `checkpoint.json` y descarga a disco las listas de operaciones y eventos acumuladas en memoria. Al arrancar, si encuentra un checkpoint y no se pidió `--reset`, lo carga y salta todas las velas cuya fecha sea $\leq$ `ts_ultimo_procesado`.

Un detalle fácil de pasar por alto: al saltar velas ya procesadas, X4 igual actualiza `cierre_previo` de cada activo. Ese diccionario guarda el último precio conocido de cada activo y se usa para valorar posiciones de activos que no tienen vela en el instante actual (una acción de EE.UU. no cotiza de madrugada, pero su posición sigue existiendo). Si no se actualizara durante el salto, el primer cálculo de patrimonio tras reanudar usaría precios de hace meses.

Las claves numéricas son el punto delicado de la serialización: JSON solo admite claves de texto, así que al guardar se convierten a texto y al cargar se reconvierten con `float(k)`. Esa conversión de ida y vuelta está en `_cargar_checkpoint` y `_guardar_checkpoint` y debe mantenerse simétrica.

2.2.3 Carga de datos

Las velas H1 se cargan una vez, completas, en memoria: un `DataFrame` por activo, ordenado por fecha, sin duplicados y recortado desde `fecha_inicio`. Se indexa por fecha para que la consulta `df.loc[ts]` sea directa. Las velas M1 se cargan igual pero sin indexar, porque no se consultan por fecha sino por posición aleatoria (Sección 2.5).

El eje temporal del bucle no es el de un activo, sino la **unión ordenada de todas las fechas de todos los activos**. Así, un instante en que solo cotiza BTCUSD existe en el bucle y se procesa solo para ese activo; el resto se valora al último precio conocido.

2.3 El algoritmo global

Con el estado y los datos ya presentados, el bucle completo cabe en una página. El Pseudocódigo 0 es el mapa: todo lo que viene después de esta subsección es la expansión de alguna de sus líneas.

Pseudocódigo 2.1. `ejecutar_backtest(cfg, reset, x5_mode)` — bucle principal

Entrada: configuración `cfg`, banderas `reset` y `x5_mode`

Salida: archivos de resultados en disco y estado final

```
1: datos_h1 ← cargar CSV horarios de todos los activos, recortados desde fecha_inicio
2: datos_m1 ← cargar CSV de minutos (puede faltar por activo)
3: si x5_mode entonces
4:   decidir reset solo: pasada nueva si no hay checkpoint, si terminó, o si hubo quiebra
5:   si reset entonces borrar el estado adaptativo del optimizador de soportes
6:   fin si
7: fin si
8: estado ← checkpoint si  $\neg$  reset y existe; si no, estado inicial con capital_inicial
9: si algún activo no tiene soportes entonces
10:   si x5_mode entonces elegir parámetros del primer tramo (Sec. 2.9.1)
11:   fin si
12:   recalcular_soportes(estado,  $t = fecha\_inicio$ ) // arranque en frío
13: fin si
14: todos_los_ts ← unión ordenada de las fechas de todos los activos
15: para cada ts en todos_los_ts hacer
16:   actualizar cierre_previo de los activos con vela en ts
17:   si ts ≤ ts_ultimo_procesado entonces omitir vuelta // ya simulada antes del checkpoint
18:   fin si
19:   si han pasado delta_recalcu_lo_soportes días desde el último recálculo entonces
20:     si x5_mode entonces elegir parámetros del nuevo tramo (explorar o explotar)
21:     fin si
22:     recalcular_soportes(estado, ts)
23:     guardar avance y omitir vuelta // la vela del recálculo no se opera
24:   fin si
25:   precios ← cierre de cada activo en ts, o su último cierre conocido
26:   para cada activo con vela en ts hacer
27:     si x5_mode entonces ctx ← fotograma de features X3 y X2 al instante ts
28:     fin si
29:     procesar_vela(ts, vela, activo, ...) // pasos A-F
30:     si x5_mode y toca registro periódico y hay posiciones abiertas entonces
31:       escribir fila 'periodico' en el store de X5
32:     fin si
33:   fin para
34:   mc ← registrar_equity(ts, estado, precios) // paso G
35:   ts_ultimo_procesado ← ts
36:   si patrimonio ≤ 0 o nivel de margen ≤ STOP_OUT_LEVEL entonces
37:     registrar quiebra, guardar todo y cortar
38:   fin si
39:   si múltiplo de 24 velas entonces guardar checkpoint y volcar operaciones y eventos a disco
40:   fin si
41: fin para
42: guardar checkpoint final, volcar pendientes, imprimir resumen
```

Tres observaciones sobre este esqueleto:

La vela del recálculo se congela.

Cuando toca recalcular soportes, X4 recalcula, guarda y salta a la siguiente vela sin operar. El motivo es de coherencia interna: el recálculo puede cancelar órdenes en espera y crear otras nuevas, y mezclar eso con la ejecución de la misma vela dejaría un orden de eventos ambiguo (¿la orden que se canceló alcanzó a ejecutarse?). Congelar la vela cuesta una hora de simulación cada varios

días y elimina la ambigüedad.

El disparo del recálculo es por umbral, no por hora exacta.

La condición es «han pasado al menos δ días», evaluada en la primera vela disponible que la cumpla. Una versión anterior exigía además una hora concreta del día, y eso hacía que activos sin sesión de 24 horas —las acciones— pudieran no tener nunca una vela a esa hora y quedaran con soportes congelados para siempre.

El orden interno de cada vela es fijo.

Primero se procesan todos los activos, después se registra el patrimonio, después se evalúa la quiebra. Registrar el patrimonio antes de operar daría una foto desfasada; evaluar la quiebra antes de registrar dejaría la última fila del historial sin escribir.

2.4 Los pasos A–F: la lógica de trading de una vela

`_procesar_candle` es el corazón. Recibe una vela de un activo y aplica seis pasos en orden estricto. Antes de ellos hace dos actualizaciones de contadores: incrementa la duración en velas de cada posición abierta y actualiza su peor pérdida flotante usando el mínimo de la vela.

Pseudocódigo 2.2. `_procesar_candle(ts, vela, activo, ...)`

Entrada: vela con Open/High/Low/Close, estado del activo

```
1: para cada posición abierta p hacer
2:   p.duracion_velas  $\leftarrow$  p.duracion_velas + 1
3:   p.drawdown_max  $\leftarrow$   $\min(\text{p.drawdown\_max}, (\text{Low} - p_{ap}) \cdot L)$ 
4: fin para
5: precio_saliente  $\leftarrow$  paso_A(...) // cancelar OE huérfanas
6: usar_iv  $\leftarrow$  necesita_intravela(...)
7: si usar_iv y hay datos de minutos entonces
8:   simular_intravela(...) // repite B–F sobre 60 velas de un minuto
9: si no
10:  paso_B(vela) // ejecutar OE alcanzadas
11:  paso_C(High) // mover stops protectores
12:  paso_D(Low) // cerrar por pérdida máxima
13:  paso_E(Low) // cerrar por stop tocado
14:  paso_F(Close) // crear OE nuevas
15: fin si
```

2.4.1 Paso A — cancelar órdenes en espera huérfanas

Cada recálculo produce un conjunto de soportes distinto. Una orden en espera colocada sobre un soporte que ya no existe debe retirarse: el nivel dejó de ser interesante. El paso A recorre las OE y elimina las que no coinciden con ningún soporte vigente.

Pseudocódigo 2.3. `_paso_A` — cancelación de OE sin soporte

Salida: precio más alto entre las canceladas, o *nada*

```
1: a_eliminar  $\leftarrow$   $\{p \in \text{OE} : p \notin \text{soportes}\}$ 
2: para cada p en a_eliminar hacer
3:   registrar evento OE_eliminada con motivo soporte_desactivado
4:   borrar OE en p
5: fin para
6: retornar  $\max(\text{a\_eliminar})$  si no está vacío
```

El valor de retorno no es decorativo. Si se acaba de retirar un buy limit que estaba razonablemente cerca del precio, la zona inmediatamente inferior queda descubierta. El paso F usa ese precio para permitir que un soporte cercano ocupe el hueco aunque no cumpla la distancia mínima habitual (Sección 2.4.6). Es la misma regla de reemplazo que aplica el sistema en vivo en `X1.crear_ordenes_espera`, y mantenerla idéntica en ambos lados es lo que hace comparables los resultados simulados con los reales.

2.4.2 Paso B — ejecutar las órdenes que el precio alcanzó

Una orden en espera a precio p se ejecuta si el precio bajó hasta p . Con velas horarias eso se traduce en $\text{Low} \leq p$. Pero hay un caso especial que merece tratamiento aparte: el hueco de mercado.

El caso normal.

Se recorren las OE con $p < \text{Open}$ de mayor a menor. Si $\text{Low} \leq p$, la orden se ejecuta al precio p exacto.

El hueco.

Si una orden estaba en \$174 y el mercado abre en \$170 —viernes cierre en \$176, lunes apertura en \$170 tras una mala noticia—, el precio nunca pasó por \$174: se lo saltó⁴. En la realidad el bróker ejecuta todas las órdenes límite pendientes por encima de la apertura al precio de apertura. X4 replica eso: detecta el conjunto $\text{oes_gap} = \{p \in \text{OE} : p \geq \text{Open}\}$ y ejecuta todas al precio $\min(\text{oes_gap})$, no al precio pedido. El resultado es una entrada peor que la planificada, que es exactamente lo que ocurre en vivo.

Pseudocódigo 2.4. `_paso_B` — ejecución de OE, con y sin hueco

```

1: oes_gap  $\leftarrow \{p \in \text{OE} : p \geq \text{Open}\}$ 
2: si oes_gap no está vacío entonces
3:   precio_gap  $\leftarrow \min(\text{oes\_gap})$ 
4:   para cada p en oes_gap ordenadas hacer
5:     margen_nueva  $\leftarrow \text{precio\_gap} \cdot \text{lote} \cdot \text{unidades} / \text{apalancamiento}$ 
6:     si margen libre – margen_nueva < MARGEN_LIBRE_MIN_BT entonces
7:       cancelar la OE con motivo margen_insuficiente; omitir vuelta
8:     fin si
9:     mover OE  $\rightarrow$  OA anotando precio_gap como precio de apertura
10:    registrar evento OE_ejecutada con es_gap = verdadero
11:  fin para
12: fin si
13: para cada p en OE con  $p < \text{Open}$ , de mayor a menor hacer
14:  si  $\text{Low} \leq p$  entonces
15:    mismo control de margen; si no alcanza, cancelar y omitir vuelta
16:    mover OE  $\rightarrow$  OA con precio de apertura  $p$ 
17:    registrar evento OE_ejecutada con es_gap = falso
18:  fin si
19: fin para
```

⁴**Hueco** (*gap*): salto de precio entre el cierre de una vela y la apertura de la siguiente, sin cotizaciones intermedias. Típico de fines de semana y de la apertura de mercados no continuos. Es la razón por la que una orden límite puede ejecutarse a un precio peor que el pedido.

El control de margen.

Antes de convertir cada OE en OA se verifica que quede margen libre⁵ suficiente. Si no lo hay, la orden no se ejecuta: se cancela y se registra el motivo. Esto reproduce el rechazo del bróker y evita que la simulación abra posiciones imposibles.

El control se hace *orden por orden*, recalculando el margen libre en cada iteración, porque cada ejecución consume margen y cambia la respuesta para la siguiente. En un hueco grande pueden ejecutarse decenas de órdenes en el mismo instante y la última no tiene por qué caer.

2.4.3 Paso C — el stop protector móvil

El mecanismo que convierte una ganancia flotante en ganancia asegurada. Tiene dos regímenes:

- **Sin stop todavía** ($sl = 0$). Se activa por primera vez solo si la ganancia flotante calculada con el máximo de la vela alcanza A : $(High - p_{ap}) \cdot L \geq A$. El stop se coloca en $High - B/L$.
- **Con stop activo**. El nuevo candidato es $High - B/L$. Se adopta *solo si es mayor* que el actual. El stop sube con el precio y nunca baja.

Pseudocódigo 2.5. `_paso_C` — trailing stop⁶

```
1: para cada posición abierta en  $p_{ap}$  hacer
2:    $ga \leftarrow (High - p_{ap}) \cdot L$  // ganancia flotante con el máximo de la vela
3:    $ganancia\_max \leftarrow \max(ganancia\_max, ga)$ 
4:   si  $sl = 0$  entonces
5:     si  $ga \geq A$  entonces
6:        $sl \leftarrow High - B/L$ 
7:       reponer una OE en  $p_{ap}$  si no existe // el nivel vuelve a quedar libre
8:       registrar evento SL_cambiado
9:     fin si
10:  si no
11:     $sl\_nuevo \leftarrow High - B/L$ 
12:    si  $sl\_nuevo > sl$  entonces
13:       $sl \leftarrow sl\_nuevo$ ; registrar evento
14:    fin si
15:  fin si
16: fin para
```

La reposición de la orden en espera.

La línea sutil de este paso: cuando una posición activa su primer stop, se vuelve a colocar una OE en su mismo precio de apertura. La lectura es que ese nivel ya demostró funcionar —el precio bajó hasta ahí, rebotó y la posición está en ganancia asegurada—, así que vale la pena tener otra orden esperando por si el precio vuelve. Sin esta línea, cada soporte serviría para una sola operación por ciclo de recálculo.

⁵**Margen:** dinero que el bróker inmoviliza como garantía al abrir una posición apalancada. Se calcula como valor de la posición dividido por el apalancamiento. **Margen libre** = patrimonio — margen usado: lo que queda disponible para abrir posiciones nuevas o absorber pérdidas.

⁶**Trailing stop** (stop dinámico o «de arrastre»): orden de venta automática cuyo precio de disparo sigue al precio de mercado mientras este sube, y se queda quieta cuando baja. Su efecto es fijar un piso creciente para el resultado de la operación.

2.4.4 Paso D — cierre por pérdida máxima

Corte de pérdidas duro. Si la pérdida flotante medida con el mínimo de la vela supera el umbral, la posición se cierra al mínimo:

$$(p_{ap} - \text{Low}) \cdot L > \text{PERDIDA_MAX_BT} \implies \text{cerrar a Low}$$

Pseudocódigo 2.6. _paso_D — cierre por pérdida máxima

```
1: a_cerrar  $\leftarrow \{p_{ap} \in \text{OA} : (p_{ap} - \text{Low}) \cdot L > \text{PERDIDA\_MAX\_BT}\}$ 
2: para cada  $p_{ap}$  en a_cerrar hacer
3:   sacar la posición de OA; actualizar su peor pérdida flotante
4:   retorno  $\leftarrow (\text{Low} - p_{ap}) \cdot L$ 
5:   capital  $\leftarrow \text{capital} + \text{retorno}$ ; GC[activo]  $\leftarrow \text{GC[activo]} + \text{retorno}$ 
6:   construir el registro de operación con motivo perdida_max y añadirlo a trades
7:   si x5_mode entonces construir la fila OC y escribirla en el store de X5
8:   fin si
9:   registrar evento posicion_cerrada
10: fin para
```

Nótese que el precio de cierre es el mínimo de la vela, no el nivel exacto donde se cruzó el umbral. Es la convención conservadora: se asume el peor punto alcanzado dentro de la vela. Cuando la simulación intra-vela está activa, el mismo paso se ejecuta sobre velas de un minuto y el precio de cierre es mucho más ajustado.

2.4.5 Paso E — cierre por stop tocado

Idéntico en estructura al paso D, pero la condición es que el mínimo de la vela haya llegado al stop, y el precio de cierre es el stop mismo (no el mínimo): una orden de stop se ejecuta a su nivel.

Pseudocódigo 2.7. _paso_E — cierre por stop protector

```
1: a_cerrar  $\leftarrow \{p_{ap} \in \text{OA} : s1 > 0 \wedge \text{Low} \leq s1\}$ 
2: para cada  $p_{ap}$  en a_cerrar hacer
3:   retorno  $\leftarrow (s1 - p_{ap}) \cdot L$  // normalmente positivo, por construcción del paso C
4:   actualizar capital y ganancia cerrada del activo
5:   registrar la operación con motivo trailing_stop y, en modo X5, la fila del store
6: fin para
```

Por qué D va antes que E.

Ambos miran el mínimo de la vela y ambos pueden aplicar a la misma posición. Poner D primero significa que, si una posición cruza a la vez su pérdida máxima y su stop, se registra como pérdida máxima. Con velas horarias es un empate raro; con simulación intra-vela activa prácticamente desaparece, porque las condiciones se evalúan minuto a minuto y una ocurre estrictamente antes que la otra.

2.4.6 Paso F — crear órdenes en espera

Cierra el ciclo: recorre los soportes de mayor a menor y coloca una orden en espera en cada uno que cumpla las condiciones.

Pseudocódigo 2.8. _paso_F — creación de OE sobre soportes libres

Entrada: precio de referencia p_{ref} (cierre de la vela), **precio_saliente** del paso A

```
1: margen_comprometido  $\leftarrow 0$ 
2: si precio_saliente existe y precio_saliente  $< p_{ref}$  entonces
```

```

3:   umbral_reemplazo  $\leftarrow (p_{ref} + \text{precio\_saliente})/2$ 
4: fin si
5: para cada soporte en orden decreciente hacer
6:   si ya hay OA u OE en soporte entonces omitir vuelta
7:   fin si
8:   distancia_ok  $\leftarrow (p_{ref} - \text{soporte}) \cdot L \geq A$ 
9:   reemplazo_ok  $\leftarrow$  umbral_reemplazo existe  $\wedge$  soporte < umbral_reemplazo
10:  si ni distancia_ok ni reemplazo_ok entonces omitir vuelta // queda en estado 00
11:  fin si
12:  margen_nueva  $\leftarrow$  soporte  $\cdot$  lote  $\cdot$  unidades / apalancamiento
13:  si margen libre - margen_comprometido - margen_nueva < MARGEN_LIBRE_MIN_BT
    entonces omitir vuelta
14:  fin si
15:  margen_comprometido  $\leftarrow$  margen_comprometido + margen_nueva
16:  crear OE en soporte; en modo X5, adjuntar el fotograma de features X3 y X2
17:  registrar evento OE_creada
18: fin para

```

La condición de distancia.

$(p_{ref} - \text{soporte}) \cdot L \geq A$ dice: solo coloca la orden si, cuando se ejecute, tendrá espacio para ganar al menos A antes de volver al precio actual. Colocar órdenes pegadas al precio produce entradas que se ejecutan enseguida y quedan atrapadas sin margen de maniobra.

La regla de reemplazo.

El punto medio entre el precio actual y el buy limit recién cancelado define una zona donde un soporte puede entrar aunque esté más cerca de lo normal. Recupera la cobertura que el paso A acaba de retirar, sin abrir la puerta a llenar toda la zona cercana de órdenes.

Margen comprometido dentro del ciclo.

Como el paso F puede crear muchas órdenes en una sola pasada y el margen de una orden en espera no está realmente reservado hasta que se ejecuta, se lleva un acumulador local **margen_comprometido**. Sin él, el paso colocaría órdenes por un total que la cuenta no puede sostener si todas se ejecutaran a la vez.

2.4.7 Por qué este orden y no otro

El orden A–F no es arbitrario. Se puede justificar como una cadena de dependencias:

1. **A antes que todo**, porque opera sobre información que acaba de cambiar (los soportes) y podría estar dejando órdenes vivas sobre niveles ya inválidos.
2. **B antes que C, D y E**, porque una orden que se ejecuta en esta vela debe poder gestionarse en esta misma vela: si el precio cae fuerte, una posición recién abierta puede tener que cerrarse de inmediato por pérdida máxima.
3. **C antes que E**, porque el stop debe estar actualizado antes de comprobar si el precio lo tocó. Al revés, se cerrarían posiciones con un stop viejo.
4. **F al final**, porque necesita el estado ya depurado: sabe qué niveles quedaron ocupados tras las ejecuciones de B y liberados tras los cierres de D y E, y cuánto margen queda realmente.

2.5 Simulación intra-vela

2.5.1 El problema

Una vela horaria dice que el precio abrió en 180, tocó 184, bajó hasta 176 y cerró en 181. No dice en qué orden. Y el orden importa muchísimo: si primero subió a 184 y luego bajó a 176, el stop protector se activó y la operación cerró en ganancia; si primero cayó a 176 y luego subió, quizá se cerró antes por pérdida máxima. Con las mismas cuatro cifras, dos resultados opuestos.

2.5.2 La solución adoptada

Cuando el desenlace de la vela depende del orden interno, X4 toma un tramo real de sesenta minutos del historial M1 del mismo activo —elegido al azar—, lo deforma para que encaje exactamente en la vela horaria y simula los pasos B–F sobre cada uno de esos sesenta minutos.

El reescalado es una transformación afín que preserva la forma del tramo pero fuerza sus extremos:

$$x' = \text{Low}_{H1} + (x - \text{Low}_{M1}) \cdot \frac{\text{High}_{H1} - \text{Low}_{H1}}{\text{High}_{M1} - \text{Low}_{M1}}$$

y luego se fija a mano la apertura de la primera vela de un minuto y el cierre de la última para que coincidan con los de la vela horaria. El resultado es un camino de precios plausible —con la microestructura de un tramo real de ese mismo activo— que respeta los cuatro valores conocidos.

Pseudocódigo 2.9. `_necesita_intravela` — ¿hace falta el detalle de minutos?

```
1: rango ← High – Low
2: para cada posición abierta hacer
3:   si s1 = 0 entonces
4:     si rango · L ≥ A o (pap – Low) · L ≥ PERDIDA_MAX_BT entonces retornar verdadero
5:   fin si
6:   si no
7:     sl_nuevo ← High – B/L
8:     si sl_nuevo > s1 y Low ≤ sl_nuevo entonces retornar verdadero
9:   fin si
10: fin si
11: fin para
12: para cada orden en espera en p hacer
13:   si Low ≤ p y (rango · L ≥ A o (p – Low) · L ≥ PERDIDA_MAX_BT) entonces retornar verdadero
14: fin si
15: fin para
16: retornar falso
```

La prueba busca exactamente las situaciones ambiguas: una vela lo bastante amplia como para que quepan dentro tanto la activación del stop como su ejecución posterior, o una posición que podría a la vez alcanzar su stop y su pérdida máxima. Cuando la vela es tranquila, no hay ambigüedad y se resuelve con los cuatro valores horarios, que es mucho más rápido.

Pseudocódigo 2.10. `_simular_intravela`

```
1: si no hay datos M1 o hay menos de 60 velas entonces retornar
2: fin si
3: t ← entero aleatorio en [0, |M1| – 60]
4: bloque ← reescalar(M1[t : t + 60], vela horaria)
5: para cada vela_m1 en bloque hacer
6:   paso_B(vela_m1); paso_C(Highm1); paso_D(Lowm1)
7:   paso_E(Lowm1); paso_F(Closem1)
```

8: **fin para**

2.5.3 Limitaciones honestas de este enfoque

- **Es aleatorio.** Dos corridas idénticas dan resultados distintos, porque el tramo de minutos elegido cambia. No hay semilla fija. Para el uso principal —generar variedad para X5— eso es más virtud que defecto, pero invalida la comparación exacta entre dos corridas del modo versión.
- **El tramo prestado no corresponde al momento simulado.** Se toma de cualquier punto del historial de minutos, no del instante que se está simulando. Aporta una microestructura realista promedio, no la real de esa hora.
- **Todas las velas del bloque comparten la marca de tiempo horaria.** Las operaciones abiertas y cerradas dentro del bloque quedan fechadas con la hora de la vela horaria. La duración en minutos de una operación intra-vela no es recuperable desde los registros.
- **Sin datos M1 se degrada en silencio.** Si falta el archivo de minutos, X4 avisa por consola y resuelve la vela con los cuatro valores horarios.

2.6 Contabilidad de la cuenta

2.6.1 Las cuatro magnitudes

En cada vela, tras procesar todos los activos, `_calcular_estado_cuenta` recorre las posiciones abiertas y produce:

$$\begin{aligned}\text{ganancia flotante} &= \sum_{\text{OA}} (p_{\text{actual}} - p_{\text{ap}}) \cdot \text{lote} \cdot \text{unidades} \\ \text{patrimonio} &= \text{saldo} + \text{ganancia flotante} \\ \text{margen usado} &= \sum_{\text{OA}} \frac{p_{\text{ap}} \cdot \text{lote} \cdot \text{unidades}}{\text{apalancamiento}} \\ \text{nivel de margen} &= \frac{\text{patrimonio}}{\text{margen usado}} \times 100\end{aligned}$$

El **saldo** solo cambia cuando una posición cierra; el **patrimonio** cambia con cada tick. El **nivel de margen**⁷ es la métrica de supervivencia.

2.6.2 Quiebra de la cuenta

X4 declara quiebra y detiene la simulación si el patrimonio cae a cero o menos, o si hay margen usado y el nivel de margen cae por debajo de `STOP_OUT_LEVEL` (50 % por defecto). Se registra un evento `stop_out` con la foto completa de la cuenta, se marca `estado['stop_out'] = True` en el checkpoint y se corta el bucle.

En modo X5 el capital inicial es de un millón de dólares virtuales por activo, deliberadamente desproporcionado. La razón: en recolección de datos no interesa saber si la cuenta sobrevive, sino observar cómo se comporta cada combinación de parámetros. Un tope de capital cortaría las pasadas antes de tiempo y sesgaría el conjunto de datos hacia configuraciones conservadoras.

⁷**Nivel de margen** (*margin level*): patrimonio dividido por margen usado, en porcentaje. Es el indicador que vigila el bróker: por debajo de cierto porcentaje empieza a cerrar posiciones por su cuenta (*margin call* y luego *stop out*). Un nivel de 1000 % es holgado; 100 % significa que el patrimonio apenas cubre la garantía exigida.

2.6.3 Los dos archivos de patrimonio

`equity_global.csv` recibe una fila por vela con saldo, patrimonio, margen usado, margen libre, nivel de margen y número de posiciones abiertas y en espera. `equity_activos.csv` recibe una fila por vela y *activo* con tres columnas: ganancia cerrada acumulada (GC), ganancia flotante actual (GA) y su suma (GT). El primero sirve para la curva de capital; el segundo para atribuir el resultado a cada activo.

2.7 Recálculo de soportes dentro del backtest

2.7.1 Delegación en X0

X4 no reimplementa el algoritmo de soportes: importa `_procesar_valor_N` de X0 y lo llama con `fecha_hora_max = ts`. Esa única bandera cambia tres comportamientos dentro de X0: recorta los datos, lee y escribe en el caché de backtesting en vez de en el archivo de producción, y omite la generación de gráficos.

2.7.2 Paralelismo por activo

Los recálculos de distintos activos son independientes, así que se reparten entre hasta cuatro procesos con un `ProcessPoolExecutor`⁸. La búsqueda de soportes es puro cálculo numérico, de modo que aquí los procesos son la elección correcta.

Pseudocódigo 2.11. `_recalcular_soportes(estado, ts, cfg, x5_mode)`

```
1: params__soporte  $\leftarrow \{K, N_{EXP}, \Lambda\}$  del tramo actual si x5_mode, si no nada
2: tareas  $\leftarrow$  una por activo, con: activo,  $N$ , rutas, ts, precios de sus posiciones abiertas
3: infos  $\leftarrow$  ejecutar tareas en paralelo (hasta 4 procesos) llamando a X0._procesar_valor_N
4: informar por consola el rango de precios usado, el arranque en caliente y la FO de cada búsqueda
5: para cada activo hacer
6:   nuevos  $\leftarrow$  leer del caché de backtesting la solución más reciente con  $t^* \leq \mathbf{ts}$ 
7:   si nuevos no está vacío entonces
8:     soportes del activo  $\leftarrow$  ordenar(nuevos)
9:   fin si
10: fin para
11: ts_ultimo_recalculo  $\leftarrow \mathbf{ts}$ 
```

2.7.3 El caché de backtesting y el arranque en caliente

Optimizar N soportes desde cero tarda minutos. Como cada recálculo es una variación pequeña del anterior —cinco días más de datos—, empezar desde la solución previa ahorra la mayor parte del trabajo. Eso es un *arranque en caliente*⁹.

El caché `{ACTIVO}_{N}_bt.json` es un diccionario de fecha a lista de soportes. Al resolver el instante t , X0 busca la clave más reciente con $t^* \leq t$ y usa esa solución como punto de partida. Al terminar, añade la solución nueva bajo la clave t .

Hay una distinción que suena parecida pero es distinta, y conviene fijarla:

- **warm_start** controla **de dónde sale la solución inicial**: del caché, o de puntos aleatorios.

⁸**Proceso** vs. **hilo**: Python no permite que varios hilos ejecuten código Python a la vez (por el bloqueo global del intérprete o GIL), así que para trabajo intensivo en cálculo hay que usar procesos separados, cada uno con su propio intérprete. `ProcessPoolExecutor` reparte tareas entre procesos; `ThreadPoolExecutor`, entre hilos —útil solo cuando el trabajo es esperar a que termine otra cosa, como en la orquestación de X5.

⁹**Arranque en caliente** (*warm start*): iniciar un algoritmo iterativo desde una solución conocida y razonablemente buena en lugar de un punto aleatorio. Reduce el número de iteraciones necesarias, a costa de sesgar la búsqueda hacia el entorno de esa solución.

- `cold_start` controla **si se hereda el paso adaptativo** `delta_inicial` del optimizador. En modo X5 se activa (es decir, no se hereda) porque cada tramo cambia K , N_{EXP} y Λ : heredar la presión acumulada sobre una función objetivo que ya no es la misma dejaría al optimizador convergido de entrada, sin mover un solo soporte.

2.7.4 Las posiciones abiertas son anclas

A la búsqueda de soportes se le pasa la lista de precios de las posiciones abiertas del activo (`ordenes_abiertas_bt`). X0 las trata como fijas: puede mover los demás soportes, pero no esos. Tiene sentido operativo —hay dinero comprometido en ese nivel exacto, moverlo en la lista no cambia la realidad— y evita que el paso A cancele órdenes que en realidad ya son posiciones.

2.8 Persistencia de operaciones y eventos

2.8.1 Los dos registros

`trades.json` contiene una entrada por operación cerrada, con todo lo necesario para analizarla después: identificador, activo, N , nivel del soporte, fechas de apertura y cierre, precios, motivo del cierre, retorno en dólares y en porcentaje del capital, duración en velas, peor pérdida flotante y mejor ganancia flotante alcanzadas, capital de la cuenta al abrir, si se resolvió intra-vela, y una copia del bloque de parámetros vigentes.

`events.json` es más granular: registra también las creaciones y cancelaciones de órdenes y cada movimiento de stop. Sirve para reconstruir la película completa cuando algo no cuadra.

2.8.2 Identificadores reconstruibles

El identificador de una operación se compone como `{activo}_{fecha_apertura}_{precio_apertura}`. No se almacena en ningún lado: se reconstruye cuando hace falta, tanto al abrir la posición como al cerrarla, y sale idéntico las dos veces porque depende solo de datos que no cambian. Es una forma barata de tener identificadores estables sin llevar un contador global ni guardar estado extra.

2.8.3 Escritura segura

Las operaciones y eventos se acumulan en listas en memoria y se vuelcan periódicamente. El volcado lee el archivo existente, le añade lo nuevo, escribe a un archivo temporal y lo renombra sobre el destino. El renombrado es atómico¹⁰: si el proceso muere a mitad, el archivo anterior queda intacto.

La lectura, además, reintenta hasta diez veces con pausas de 0,2 segundos. No es paranoia gratuita: el proyecto vive en una carpeta sincronizada por OneDrive en Windows, y el servicio de sincronización bloquea temporalmente los archivos recién escritos. Sin el reintento, el volcado siguiente fallaba con un error de permisos.

2.9 Modo X5: X4 al servicio de la recolección

Todo lo anterior describe X4 como simulador. Esta subsección describe qué se le añade cuando corre con `--x5`. Son cuatro cosas.

2.9.1 Uno: los parámetros cambian durante la corrida

En modo versión, K , N_{EXP} , Λ , A , B , N y `PERDIDA_MAX` son constantes. En modo X5 se regeneran cada `delta_recalculo_soportes` días simulados, acoplados al recálculo de soportes: primero se eligen los parámetros nuevos, después se recalculan los soportes con ellos.

¹⁰**Escritura atómica:** técnica para que un archivo nunca quede a medio escribir. Se escribe primero una copia temporal completa y luego se renombra sobre el original; el sistema operativo garantiza que el renombrado ocurre entero o no ocurre. Si el programa muere a mitad, el archivo original sigue intacto.

Pseudocódigo 2.12. `_seleccionar_params_x5` — explorar o explotar

```
1: para cada activo hacer
2:   (tipo, modelo)  $\leftarrow$  modelo cargado al inicio del ciclo
3:   es_explore  $\leftarrow$  (modelo no existe) o (aleatorio[0,1) < EXPLORATION_RATE)
4:   si es_explore entonces
5:     parámetros  $\leftarrow$  sorteo uniforme dentro de X5_PARAM_RANGES del activo
6:      $B \leftarrow \alpha \cdot A$  con  $\alpha \sim U(0,3; 0,7)$  // B debe quedar por debajo de A
7:   si no
8:     ctx  $\leftarrow$  contexto de mercado al instante ts (features X3, X2, temporales, cartera)
9:     parámetros  $\leftarrow$  X5.inferir_con_contexto(activo, tipo, modelo, ctx, rangos)
10:  fin si
11: fin para
12: retornar parámetros y etiqueta (EXPLORE / EXPLOIT) por activo
```

Esta es la disyuntiva clásica entre **explorar** (probar combinaciones nuevas para aprender) y **explotar** (usar lo que el modelo ya cree que es mejor). La proporción la fija `EXPLORATION_RATE = 0,30`: incluso con modelo entrenado, tres de cada diez tramos se sortean al azar.

Mantener exploración permanente no es un capricho. Si X4 solo usara las recomendaciones del modelo, el conjunto de datos se llenaría de filas parecidas entre sí y el modelo aprendería cada vez mejor una región cada vez más chica del espacio, sin manera de descubrir que otra región es mejor. Es el sesgo de selección observacional: solo se observa lo que se decidió observar.

El muestreo de B merece un comentario. A es la ganancia mínima que activa el stop y B la distancia a la que el stop sigue al precio. Si $B > A$, el primer stop quedaría por debajo del precio de entrada y podría cerrar la posición en pérdida pese a haber alcanzado la ganancia de activación. Sortearlos de forma independiente produciría muchas combinaciones incoherentes; por eso B se sortea como una fracción de A entre 0,3 y 0,7.

Una vez elegidos, `_aplicar_params_x5` los inyecta mutando el objeto de configuración en memoria. Los escalares (K , N_{EXP} , Λ , A , B , pérdida máxima) toman el valor del primer activo de la lista; los que son por activo (N y lote) se asignan uno a uno. Como en modo X5 cada proceso corre un solo activo, ese colapso a escalar es exacto, no una aproximación.

2.9.2 Dos: cada operación cerrada se convierte en una fila de entrenamiento

En cada vela y activo se calcula un fotograma de contexto (`_compute_x5_snapshot`): indicadores técnicos de X3 sobre los datos hasta ese instante, y score fundamental de X2 correspondiente a esa fecha. Ese fotograma se adjunta a la orden en dos momentos:

- cuando se **crea** la orden en espera, como bloque `x3_oe / x2_oe`;
- cuando se **ejecuta** y pasa a posición abierta, como bloque `x3_oa / x2_oa`.

Al cerrarse la posición, `_construir_fila_oc` arma la fila completa: parámetros vigentes, features temporales de los dos momentos, los dos bloques de contexto, el estado de la cartera al cerrar y, finalmente, los objetivos que el modelo debe aprender a predecir.

Pseudocódigo 2.13. `_construir_fila_oc` — de operación cerrada a fila de entrenamiento

Entrada: posición cerrada, registro de operación, estado del activo, configuración

- 1: **fila** $\leftarrow$ {**tipo_registro**: 'oc', activo, fechas de OE, OA y OC}
- 2: añadir parámetros vigentes: N , K , N_{EXP} , Λ , A , B , multiplicador de lote, pérdida máxima
- 3: añadir features temporales del instante de la OE (hora, día, mes, festivos, indicadores binarios)
- 4: añadir `x2_*` y `x3_*` del instante de la OE
- 5: añadir contexto de cartera: número de posiciones abiertas y en espera, exposición, media y desviación de retornos abiertos, media de las últimas 5 operaciones cerradas, resultado flotante

- 6: añadir features temporales, `x2_*` y `x3_*` del instante de la OA, con sufijo `_oa`
- 7: **objetivos:** `retorno_pct`, `pnl_flotante_activo`, `pnl_cerrado_activo`
- 8: **retornar fila**

2.9.3 Tres: filas periódicas

Si solo se registraran operaciones cerradas, el conjunto de datos tendría un sesgo grave: durante una caída prolongada casi nada se cierra —las posiciones quedan abiertas acumulando pérdida flotante— y el modelo no vería esos períodos. Para cubrirlos, cada `X5_FREQ_REGISTRO_PERIODICO` velas (4 por defecto) con al menos una posición abierta se escribe una fila `'periodico'` con el mismo contexto pero sin datos de OE ni de OA, cuyo objetivo relevante es el resultado flotante del activo.

2.9.4 Cuatro: aislamiento por activo

Con `--activo BTCUSD`, X4 reduce la lista de activos a uno y redirige sus carpetas de trabajo a `bt_-BTCUSD`. Así, seis procesos pueden correr a la vez sin pisarse el checkpoint, la curva de patrimonio ni el caché de soportes. Los parámetros que la configuración de X5 define por activo (A , B , pérdida máxima) se colapsan al valor de ese activo, porque el núcleo de X4 los espera como escalares.

La decisión de reiniciar o continuar también cambia en este modo: X4 la toma solo, sin que el llamador la imponga. Empieza una pasada nueva si no hay checkpoint, si la pasada anterior llegó al final de los datos, o si terminó en quiebra; retoma el checkpoint si quedó a mitad de camino, es decir, si el proceso se interrumpió. Así una pasada completa sigue siendo una unidad independiente —con su propia secuencia de parámetros— sin tirar a la basura horas de cómputo por una interrupción.

2.10 Parámetros de X4 y su efecto

Parámetro	Valor típico	Qué controla y qué pasa al subirlo
<code>capital_inicial</code>	3 000 / 1 000 000	Saldo de partida. En modo X5 se pone absurdamente alto para que el margen nunca corte la pasada.
<code>PERDIDA_MAX_BT</code>	120 USD	Corte de pérdidas por posición. Subirlo deja correr las perdedoras: menos cierres en rojo pero peores cuando llegan.
<code>MARGEN_LIBRE_MIN_BT</code>	50 USD	Colchón mínimo de margen. Subirlo hace la simulación más conservadora: rechaza más órdenes.
<code>STOP_OUT_LEVEL</code>	50 %	Nivel de margen al que se declara la quiebra.
<code>delta_recalculo_soportes</code>	1 (versión) / 5 (X5)	Días entre recálculos. Bajarlo da soportes más frescos y muchísimo más cómputo; en modo X5 también acelera el cambio de parámetros.
A	2–20 USD	Ganancia que activa el stop y distancia mínima para colocar una orden. Subirlo: menos órdenes, más lejos, operaciones más ambiciosas.
B	0,3A–0,7A	Holgura del stop. Subirlo tolera más retrocesos antes de cerrar; bajarlo asegura antes y corta más operaciones prematuramente.
N	40–200	Soportes activos. Subirlo cubre más rango con entradas más finas y fragmenta más el capital.
<code>LOTAJES_M</code>	1	Multiplicador del lote mínimo. Fijo en 1 en todo el pipeline de X5.

Tabla 5: Parámetros de X4. Los siete últimos son precisamente los que X5 explora y optimiza.

3 X5 — Cerebro macro

3.1 Propósito

3.1.1 El problema: una configuración fija para un mercado que no lo es

Todos los parámetros del sistema —cuántos soportes mantener, cuánta ganancia exigir antes de proteger, cuánto dejar respirar al stop, cuánta pérdida tolerar— están hoy escritos en `config.py` como números fijos. Funcionan razonablemente en promedio, y ese promedio es el problema: el mercado de septiembre no es el de marzo, y BTCUSD no se comporta como GOOGL.

La pregunta que da origen a X5 es directa: *dado cómo está el mercado ahora mismo, ¿qué combinación de parámetros habría rendido mejor en situaciones parecidas?* Responderla a mano es inviable —son ocho parámetros continuos, seis activos y un contexto que cambia todo el tiempo—, así que se convierte en un problema de aprendizaje.

3.1.2 La idea: un modelo sustituto

Lo que de verdad interesa evaluar es una función carísima:

$$\text{RENDIMIENTO}(\text{contexto}, \text{parámetros}) = \text{resultado de operar así en esa situación}$$

Evaluarla una vez en la realidad cuesta semanas; evaluarla con X4 cuesta minutos u horas. Optimizarla directamente —probar miles de combinaciones y quedarse con la mejor— es inviable en vivo.

La salida es entrenar un **modelo sustituto**¹¹: un modelo que aprende a predecir el resultado sin correr la simulación. Una vez entrenado, buscar los mejores parámetros es barato: se le pregunta al modelo miles de veces con distintas combinaciones y se elige la que predice mejor resultado.

En una frase: **X4 evalúa la función cara unas cuantas miles de veces; X5 aprende a imitarla y después la optimiza.**

3.1.3 El ciclo cerrado

	Etapas	Qué ocurre
1	Recolectar	X5 lanza X4 en modo <code>--x5</code> . X4 simula, cambia parámetros cada pocos días y escribe una fila por operación cerrada en el store del activo.
2	Entrenar	Cuando el store supera el umbral, X5 entrena un modelo por activo con todas sus filas.
3	Inferir	Con el modelo entrenado se busca la combinación de parámetros que maximiza el resultado predicho para el contexto actual.
4	Realimentar	Esa recomendación se usa en los tramos «explotar» de la siguiente pasada de X4 (y, en Fase 2, en el trading real). Vuelve a 1.

Tabla 6: El bucle de X5. Cada vuelta mejora el modelo con datos que el propio modelo ayudó a generar.

3.1.4 Las dos fases de despliegue

Fase 1 (actual, `TIPO_EJECUCION = "est"`).

Todo el bucle vive en simulación. X4 llena los stores, X5 entrena y escribe sus recomendaciones en `config/active_parameters.json`. El sistema en vivo (X1) **no lee ese archivo**: sigue con los valores fijos de `config.py`. Es la fase de acumulación de evidencia.

¹¹**Modelo sustituto** (*surrogate model*): modelo estadístico barato que imita una función cara de evaluar. Se ajusta con un conjunto de evaluaciones reales de la función original y luego se usa como reemplazo para optimizar, porque consultarlo cuesta milisegundos. Es la misma idea detrás de la optimización bayesiana.

Fase 2 (TIPO_EJECUCION = "din").

X1 lee `active_parameters.json` y opera con los parámetros recomendados por activo, y cada operación real cerrada alimenta directamente el store. El bucle se cierra con datos reales. La transición es **por activo**: un activo cuyo estado sea `untrained` cae de vuelta a los valores de `config.py` de forma automática, así que BTCUSD puede estar en modo dinámico mientras TSLA sigue en estático.

3.2 Interfaz de línea de comandos

X5 tiene tres modos mutuamente excluyentes y una casilla opcional:

Invocación	Efecto
<code>--recolectar</code>	Genera datos con X4 y entrena solo. Pregunta al inicio si recolectar para todos los activos en paralelo o para uno en recorrido guiado.
<code>--recolectar --demo</code>	Recorrido guiado de un activo sobre un store y un modelo aislados con sufijo <code>_demo</code> , con pausas explicativas. No toca los datos reales.
<code>--infer</code>	Recomienda parámetros con los modelos actuales y escribe <code>active_parameters.json</code> .
<code>--infer --train</code>	Reentrena desde el store y después recomienda.
<code>--status</code>	Diagnóstico: cuántas operaciones cerradas hay por activo, qué modelo corresponde y qué dice el archivo de parámetros activos.
<code>--activo BTCUSD</code>	Restringe la operación a un activo.

Tabla 7: Modos de X5. `--train` no es un modo: es una casilla que solo tiene efecto junto a `--infer`.

3.3 El store: el conjunto de datos

3.3.1 Un archivo por activo

`resources/x5/{ACTIVO}_store.csv` acumula todas las filas generadas para ese activo, a lo largo de todas las pasadas de todas las corridas. No se sobrescribe: crece. Cada fila es una observación de la forma «*en este contexto, con estos parámetros, pasó esto*».

La separación por activo es deliberada: cada activo entrena su propio modelo. Un modelo único para los seis tendría más datos, pero mezclaría regímenes de volatilidad incomparables (BTCUSD se mueve varios puntos porcentuales en horas; GOOGL no).

3.3.2 Los dos tipos de fila

- `tipo_registro = 'oc'` — se escribe cuando una posición se cierra. Es la observación completa: contexto en el momento de crear la orden, contexto en el momento de ejecutarla, parámetros vigentes y resultado.
- `tipo_registro = 'periodico'` — se escribe cada cuatro velas mientras haya al menos una posición abierta. No tiene datos de OE ni de OA; su valor está en registrar el resultado flotante durante períodos en que nada se cierra.

3.3.3 Los bloques de columnas

El orden de las columnas está fijado por `_feature_cols_de_store` y se guarda junto al modelo. Esto es esencial: si mañana el store gana una columna nueva, un modelo entrenado antes seguiría leyendo su vector en el orden con el que fue entrenado.

Que el contexto aparezca dos veces —al crear la orden y al ejecutarla— tiene una razón concreta: entre esos dos momentos pueden pasar horas o semanas, y lo que ocurrió en medio importa. Una

#	Bloque	Prefijo	Contenido
1	Parámetros	—	n_ejecucion, K, N_EXP, LAMBDA, A, B, LOTAJES_M, PERDIDA_MAX. Las ocho variables de decisión.
2	Temporal OE	—	Hora, día de la semana, día del mes, mes; indicadores binarios de día y mes; días hasta y desde el próximo festivo bursátil de EE.UU.
3	Fundamental OE	x2_	Score global, score comparativo entre activos, score de tendencia y componentes desagregados.
4	Técnico OE	x3_	Medias móviles, RSI, MACD, ATR, bandas de Bollinger, momento, volatilidad, caída desde máximos, pendiente de tendencia, distancia y densidad de soportes.
5	Cartera	—	Posiciones abiertas y en espera, exposición en dólares, media y desviación de los retornos abiertos, media de las últimas cinco operaciones cerradas, resultado flotante.
6–8	Contexto OA	sufijo _oa	Los bloques 2, 3 y 4 repetidos, tomados en el instante en que la orden se ejecutó.

Tabla 8: Los bloques del vector de entrada, en orden. Suman del orden de 240 columnas.

orden colocada en calma que se ejecuta durante un desplome es una operación muy distinta de una colocada y ejecutada el mismo día.

3.3.4 Los tres objetivos

X5 no predice un solo número, sino tres:

Nombre	Columna	Qué mide
retorno	retorno_pct	Resultado de <i>esta</i> operación como fracción del capital. Es el objetivo principal: el que se maximiza en inferencia.
flotante	pnl_flotante_activo	Resultado no realizado del conjunto de posiciones abiertas del activo. Único objetivo disponible en las filas periódicas.
cerrado	pnl_cerrado_activo	Resultado realizado acumulado del activo hasta ese momento.

Tabla 9: Objetivos del modelo. Los tres se entrenan; solo el primero guía la optimización.

¿Por qué entrenar tres si solo se optimiza uno? Porque los otros dos actúan como **tareas auxiliares**: obligan a la parte compartida del modelo a construir representaciones que expliquen también la salud global de la cartera, no solo el desenlace de una operación aislada. En el modelo neuronal esto es explícito (un tronco compartido y tres cabezas de salida) y su peso en la función de pérdida es menor: $l_{total} = l_{ret} + 0,3l_{flot} + 0,3l_{cerr}$.

3.3.5 Escritura desde X4

_append_x5_store añade la fila al CSV en modo *append*, escribiendo la cabecera solo si el archivo no existía. Antes de escribir comprueba que el archivo termine en salto de línea: si una ejecución anterior se cortó a mitad de fila, sin ese control la fila nueva se concatenaría a la trunca y corrompería el archivo. En la lectura, el complemento de esa defensa es on_bad_lines='skip': una fila malformada se descarta en vez de abortar la carga completa del store.

3.4 Recolección

3.4.1 El orquestador

Pseudocódigo 3.1. _recolectar() — recolección paralela

```
1: cfg_x5 ← cargar scripts/config_x5_default.py
```

```

2: activos  $\leftarrow$  cfg_x5.valores; n_ciclos  $\leftarrow$  cfg_x5.N_CICLOS_BT
3: si hay terminal interactiva entonces
4:   para cada activo hacer preguntar si reiniciarlo desde cero; si sí, borrar todo lo suyo
5:   fin para
6: fin si
7: imprimir estado inicial: operaciones cerradas por activo
8: lanzar un hilo por activo con ThreadPoolExecutor
9: para cada worker que termina hacer
10:   informar cuántas operaciones cerradas ganó ese activo
11: fin para
12: mostrar el estado final (--status)

```

El paralelismo aquí es de **hilos**, no de procesos, y no contradice lo dicho en la Sección 2.9: cada hilo no calcula nada, solo lanza un subproceso de X4 y espera a que termine. El trabajo pesado ocurre en esos seis procesos hijos independientes.

3.4.2 El worker de un activo

Pseudocódigo 3.2. `_worker_recolectar_activo(activo, x4_path, n_ciclos)`

```

1: n_ini  $\leftarrow$  operaciones cerradas en el store del activo
2: para ciclo = 1 ... n_ciclos hacer
3:   ejecutar python X4_backtester.py --x5 --activo {activo} capturando su salida
4:   n  $\leftarrow$  operaciones cerradas en el store
5:   si el subproceso falló entonces imprimir las últimas líneas de error y cortar
6:   fin si
7:   si n  $\geq$  X5_MIN_TRADES_TRAIN entonces
8:     entrenar(activo) // los tramos de explotación del ciclo siguiente ya usan este modelo
9:   fin si
10:  si n  $\geq$  X5_MIN_TRADES_FTT entonces cortar // ya hay datos de sobra
11:  fin si
12: fin para
13: retornar activo y su recuento final

```

Cada iteración de este bucle es un **ciclo**. Un ciclo lanza X4 una vez; X4 decide por su cuenta si eso significa empezar una pasada nueva desde `fecha_inicio` o retomar una interrumpida. La consecuencia práctica: el reentrenamiento ocurre **entre** ciclos, nunca durante uno. Dentro de un ciclo el modelo está congelado —se carga una sola vez al principio, en `ejecutar_x5_ciclo`—, lo cual mantiene la simulación coherente: todos los tramos de explotación de una pasada consultan la misma versión del modelo.

3.4.3 El recorrido guiado

`_recolectar_guiado` corre los ciclos de forma **secuencial** —no paralela— con la salida de X4 heredada, de modo que un narrador (`x5_demo.py`) pueda explicar cada suceso nuevo la primera vez que aparece. Los sucesos tienen identificadores estables:

El mismo mecanismo tiene dos usos según dos variables de entorno:

- `--recolectar --demo`: sufijo `_demo` activo. El store, el modelo, el historial de métricas, el checkpoint del backtest y los gráficos van a rutas paralelas. Se pausa en cada suceso nuevo. Los datos reales quedan intactos.
- `--recolectar` con alcance «uno»: sin sufijo. Misma narración y mismos gráficos, pero sobre los datos reales y **sin pausas** —una recolección oficial no puede quedarse esperando que alguien pulse una tecla.

ID	Tipo	Suceso
D01	recurrente	Recálculo de soportes (arranque en frío o periódico)
D02	recurrente	Tramo de exploración: parámetros sorteados al azar
D03	recurrente	Orden en espera creada
D04	recurrente	Orden ejecutada: pasa a posición abierta
D05	recurrente	Posición cerrada: se añadió una fila al store
D06	hito	Tramo de explotación: primera inferencia con el modelo
D07	recurrente	Avance de mes dentro del backtest
D08	hito	Se cruzó el umbral de entrenamiento; se entrena el modelo
D09	hito	Fin de la recolección del activo
D10	recurrente	Gráfico de la búsqueda de soportes recién guardado

Tabla 10: Sucesos del narrador. Los marcados como hito vuelven a pausar aunque ya se hayan visto.

3.4.4 Reiniciar un activo

`_borrar_checkpoints_activo` borra absolutamente todo lo asociado al activo dentro de su espacio de nombres: store, estado del narrador, carpeta del modelo, historial de métricas, carpeta completa del backtest (checkpoint, curvas de patrimonio, caché de soportes, registros) y gráficos. El significado es literal: como si el activo nunca se hubiera procesado.

El mismo alcance aplica en el espacio de nombres real, donde el borrado destruye datos acumulados durante horas de cómputo. La pregunta de reinicio se hace explícitamente al inicio y por activo, y solo si hay terminal interactiva: en ejecución automatizada no se pregunta nada y no se borra nada.

Cada borrado se hace con reintentos y espera creciente (0,5s, 1s, 2s, 4s, 5s...hasta unos 35 segundos en total). En Windows, OneDrive y el antivirus bloquean archivos recién escritos durante bastante más que un par de segundos, y sin ese reintento el reinicio fallaba con un error de permisos.

3.5 Entrenamiento

3.5.1 Qué modelo, según cuántos datos haya

X5 elige el modelo por volumen de datos, contando solo filas de tipo 'oc':

Operaciones cerradas	Modelo	Razón
< 500	untrained	Sin datos suficientes. Se devuelven los valores de <code>config.py</code> tal cual.
500...4999	LightGBM	Los métodos basados en árboles son muy eficientes con pocos datos tabulares y no necesitan normalización.
≥ 5000	FT-Transformer	Con suficientes datos, una red neuronal captura interacciones que los árboles aproximan a saltos.

Tabla 11: Selección automática de modelo (`_seleccionar_tipo`).

3.5.2 Preparación del vector de entrada

Pseudocódigo 3.3. `_preparar_features(store, columnas, objetivo, tipos)`

Entrada: store completo, lista ordenada de columnas, nombre del objetivo, tipos de fila admitidos

Salida: matriz X , vector y , vector de pesos w

1: $df \leftarrow$ filas del store cuyo `tipo_registro` esté en **tipos**

2: descartar filas cuyo objetivo no sea numérico // p.ej. *retorno_pct* vacío en filas periódicas

```

3:  $X \leftarrow$  matriz  $|df| \times |\text{columnas}|$ ; cada columna convertida a número, ausentes y no numéricos a 0
4:  $y \leftarrow$  columna objetivo como número
5: si X5_WINDOW_TRAIN está definido entonces quedarse con las últimas X5_WINDOW_TRAIN filas
6: fin si
7:  $w_i \leftarrow \exp(\lambda_{decay}(i - n + 1))$  para  $i = 0 \dots n - 1$ 
8:  $w \leftarrow w/\overline{w}$  // media 1, para no alterar la escala de la pérdida
9: retornar  $X, y, w$ 

```

La ponderación temporal.

La última fila del store pesa ≈ 1 y las anteriores decaen exponencialmente hacia atrás con $\lambda = 0,001$; a unas 700 filas de distancia el peso es la mitad. La intención es que el modelo escuche más al pasado reciente sin descartar el antiguo. La normalización posterior a media 1 evita que el tamaño del store cambie la magnitud efectiva de la función de pérdida.

Los ausentes van a cero.

Es una decisión discutible y conviene tenerla presente: una columna vacía y una columna que vale cero son indistinguibles para el modelo. Con los árboles el efecto es menor; en el modelo neuronal, un cero tras normalizar cae en la media de la columna, que es un valor neutro razonable.

3.5.3 LightGBM: tres modelos independientes

LightGBM¹² entrena un modelo separado por objetivo, guardados como tres archivos distintos más un cuarto con la lista ordenada de columnas.

Pseudocódigo 3.4. `_entrenar_lgbm(activo, store)`

```

1: columnas  $\leftarrow$  orden canónico de features del store
2: para cada objetivo en {retorno (solo oc), flotante (oc y periodico), cerrado (solo oc)} hacer
3:    $X, y, w \leftarrow$  preparar_features(store, columnas, objetivo, tipos)
4:   si  $|X| < 20$  entonces avisar y omitir vuelta
5:   fin si
6:   corte  $\leftarrow \max(10, \lfloor 0,8 |X| \rfloor)$  // división temporal, sin mezclar
7:   entrenar con  $X[:\text{corte}]$  ponderado por  $w$ , validando en  $X[\text{corte} :]$ 
8:   si el conjunto de validación tiene al menos 10 filas entonces activar parada temprana con paciencia 50
9:   fin si
10:  calcular métricas en entrenamiento y validación; guardar el modelo
11: fin para
12: guardar la lista de columnas y el historial de métricas

```

La división es temporal, no aleatoria.

El primer 80 % de las filas —las más antiguas— entrena; el último 20 % valida. Barajar sería un error: dejaría filas del futuro en entrenamiento y filas del pasado en validación, y la métrica resultante sería optimista sin serlo de verdad.

¹²**LightGBM**: implementación de *gradient boosting* sobre árboles de decisión. El método construye cientos de árboles pequeños en secuencia, donde cada árbol nuevo se entrena para corregir el error que dejaron los anteriores. Es el estándar de facto en datos tabulares por su precisión y velocidad, y no requiere normalizar las variables.

Parada temprana.

Se entrenan hasta mil árboles, pero si el error de validación no mejora durante cincuenta rondas seguidas, se detiene y se conserva la mejor iteración¹³.

El objetivo flotante usa más filas.

Es el único que se entrena con filas 'oc' y 'periodico' juntas, porque es el único objetivo presente en ambos tipos. Ahí es donde entra la información de los períodos bajistas prolongados en que nada se cierra.

3.5.4 FT-Transformer: un tronco y tres cabezas

Cuando el store supera las 5 000 operaciones cerradas se pasa a una red neuronal de tipo *Feature Tokenizer + Transformer*, con los hiperparámetros del artículo original (Gorishniy et al., 2021): dimensión interna $d = 192$, tres bloques, ocho cabezas de atención y red interna de 256 unidades. Con unas 240 columnas de entrada resultan alrededor de 700 000 parámetros, que se entrenan sin dificultad en una máquina normal.

La arquitectura tiene tres partes:

Tokenizador de features.

Cada columna es un solo número, y un *transformer*¹⁴ trabaja sobre secuencias de vectores. El tokenizador convierte cada número escalar x_j en un vector de dimensión d mediante pesos propios de esa columna: $t_j = x_j \cdot W_j + b_j$. Así, «RSI vale 30» y «hora vale 30» se convierten en vectores completamente distintos.

Bloques transformer.

Tres bloques con normalización previa: normalización de capa¹⁵, atención de ocho cabezas, conexión residual; luego normalización, red de dos capas con activación GELU¹⁶, conexión residual.

Agregación y cabezas.

Los 240 vectores de salida se promedian en uno solo (*mean pooling*) y de ahí salen tres capas lineales, una por objetivo.

Pseudocódigo 3.5. _entrenar_ftt(activo, store)

```
1:  $X, y_{ret}, w \leftarrow \text{preparar\_features}(\text{store}, \text{columnas}, \text{retorno\_pct}, (\text{oc}))$ 
2: si  $|X| < 50$  entonces avisar y retornar falso
3: fin si
4:  $y_{flot}, y_{cerr} \leftarrow$  los otros dos objetivos sobre las mismas filas
5: corte  $\leftarrow \max(10, \lfloor 0,8n \rfloor)$ 
6: media, desv  $\leftarrow$  estadísticos por columna calculados solo sobre  $X[: \text{corte}]$ 
7:  $X_{norm} \leftarrow (X - \text{media})/\text{desv}$  // se guardan para poder normalizar igual en inferencia
```

¹³**Parada temprana** (*early stopping*): mecanismo para evitar el sobreajuste. Se vigila el error sobre datos que el modelo no está usando para aprender; en cuanto ese error deja de mejorar, se detiene el entrenamiento aunque quedaran rondas por hacer. **Sobreajuste** (*overfitting*): cuando el modelo memoriza el ruido del conjunto de entrenamiento y por eso rinde peor con datos nuevos.

¹⁴**Transformer**: arquitectura de red neuronal basada en el mecanismo de **atención**, que permite a cada elemento de una secuencia mirar a todos los demás y decidir a cuáles hacer caso. Se popularizó en procesamiento de lenguaje — cada palabra atiende al resto de la frase— y aquí se aplica a columnas de una tabla: cada variable puede condicionar su efecto al valor de las otras.

¹⁵**Normalización de capa** (*LayerNorm*): reescala los valores dentro de cada vector para que tengan media cero y desviación uno. Estabiliza el entrenamiento evitando que las magnitudes crezcan o se desvanezcan al atravesar muchas capas. La variante *pre-norm* —normalizar antes de cada subcapa, no después— es la que mejor se comporta en redes profundas.

¹⁶**GELU**: función de activación suave, variante moderna de ReLU. Introduce la no linealidad sin la que la red colapsaría a una simple combinación lineal.

```

8: inicializar la red y el optimizador AdamW (tasa  $10^{-3}$ , regularización  $10^{-4}$ )
9: para época = 1...50 hacer
10:   para cada lote de hasta 256 filas, en orden aleatorio hacer
11:      $(\hat{y}_{ret}, \hat{y}_{flot}, \hat{y}_{cerr}) \leftarrow \text{red}(X_{lote})$ 
12:      $l \leftarrow w(\hat{y}_{ret} - y_{ret})^2 + 0,3 w(\hat{y}_{flot} - y_{flot})^2 + 0,3 w(\hat{y}_{cerr} - y_{cerr})^2$ 
13:     retropropagar, recortar el gradiente a norma 1, dar un paso del optimizador
14:   fin para
15: fin para
16: guardar pesos, número de columnas y estadísticos de normalización; calcular métricas

```

Dos detalles del pseudocódigo merecen énfasis:

- **Los estadísticos de normalización se calculan solo con el tramo de entrenamiento** y se guardan junto al modelo. Calcularlos sobre todo el conjunto sería una fuga de información sutil: la media del conjunto de validación entraría en el preprocesamiento.
- **El recorte de gradiente a norma 1** evita que un lote atípico —una operación con un retorno extremo— produzca un paso gigante que desestabilice el entrenamiento.

Limitación conocida: como las tres cabezas comparten el mismo tensor de entrada, el FT-Transformer entrena los tres objetivos únicamente sobre filas 'oc'. A diferencia de LightGBM, no aprovecha las filas periódicas para el objetivo flotante. Aprovecharlas exigiría un paso separado, y está anotado como pendiente.

3.5.5 Métricas y su historial

Tras cada entrenamiento se calculan cuatro métricas por objetivo, sobre entrenamiento y sobre validación: error absoluto medio (MAE), error cuadrático medio (MSE), error porcentual absoluto medio (MAPE) y coeficiente de determinación R^2 ¹⁷.

Todo se acumula como historial en `resources/x5/Performance/{ACTIVO}_performance.json`: una entrada por entrenamiento, con marca de tiempo, tipo de modelo y tamaños de los dos conjuntos. Permite ver si el modelo mejora conforme el store crece, que es la única señal real de que el bucle está funcionando.

3.6 Inferencia: de un modelo entrenado a una recomendación

Aquí se invierte la pregunta. En entrenamiento se busca la función; en inferencia la función está fija y se buscan las entradas que la maximizan:

$$\text{parámetros}^* = \arg \max_{\text{parámetros} \in \text{rangos}} \widehat{\text{retorno}}(\text{contexto fijo}, \text{parámetros})$$

El contexto es un dato, no una variable: describe el mercado tal como está. Las ocho columnas de parámetros son las únicas que se mueven.

3.6.1 El contexto

Hay dos formas de construirlo, con el mismo esquema de columnas:

¹⁷**Coficiente de determinación (R^2):** fracción de la variabilidad del objetivo que el modelo explica. Vale 1 si acierta perfectamente y 0 si no aporta nada sobre predecir siempre la media. Puede ser **negativo**, y en este dominio es lo esperable al principio: significa que el modelo predice peor que la media constante. El MAPE, por su parte, se calcula solo sobre filas cuyo valor real no sea prácticamente cero, porque dividir por algo minúsculo lo dispara.

- **En vivo** (`_leer_contexto_actual`): features temporales de ahora, último score fundamental de X2, última fila de features técnicas de X3. Las columnas de cartera se ponen a cero —en Fase 1 no hay cartera viva que consultar— y las columnas con sufijo `_oa` se rellenan copiando las de OE, bajo el supuesto de que la orden se ejecutará pronto.
- **Dentro del backtest** (`_contexto_inferencia_x5`, en X4): idéntico esquema, pero con datos recortados al instante simulado y con la cartera **real** de ese momento.

Que ambos produzcan exactamente las mismas claves no es un detalle estético: si difirieran, el vector de inferencia no encajaría con el orden de columnas con el que se entrenó, y el modelo leería valores en posiciones equivocadas sin dar ningún error. Por eso las funciones `_extraer_x2` de X4 y de X5 están duplicadas con una advertencia explícita de mantenerlas sincronizadas.

3.6.2 Optimización sobre LightGBM: búsqueda bayesiana

Un modelo de árboles no es derivable, así que no se puede «subir por el gradiente». Se usa Optuna¹⁸, con 200 intentos por inferencia.

Pseudocódigo 3.6. `_inferir_lgbm` — búsqueda bayesiana de parámetros

Entrada: modelo del objetivo `retorno`, contexto fijo, orden de columnas

- 1: **función** OBJETIVO(intento)
- 2: $p \leftarrow$ sortear los ocho parámetros dentro de sus rangos según lo que sugiera el intento
- 3: *// LAMBDA se sugiere en escala logarítmica: su rango es pequeño y positivo*
- 4: **vector** $\leftarrow$ contexto fijo, con las ocho posiciones de parámetros reemplazadas por p
- 5: **retornar** predicción del modelo sobre **vector**
- 6: **fin función**
- 7: **estudio** $\leftarrow$ crear estudio de maximización con muestreador TPE y semilla fija
- 8: ejecutar `X5_N_OPTUNA_TRIALS` intentos
- 9: **retornar** los mejores parámetros, redondeados y con los enteros convertidos

La semilla fija (42) hace la recomendación reproducible: mismo contexto y mismo modelo dan siempre el mismo resultado.

3.6.3 Optimización sobre el FT-Transformer: ascenso por gradiente

Una red neuronal sí es derivable respecto de su entrada, y eso permite una técnica más directa: tratar los parámetros como variables a optimizar y subir por el gradiente¹⁹.

El obstáculo de implementación es que solo ocho de las 240 posiciones del vector deben ser variables; las otras 232 son constantes. La solución es construir el vector de entrada como

$$x = \text{contexto}_{\text{norm}} + P^{\top} p$$

donde P es una matriz constante de ceros y unos que coloca cada parámetro en su posición, y p es el vector de ocho variables. Como P es constante, el gradiente fluye limpiamente hacia p y solo hacia p .

Pseudocódigo 3.7. `_inferir_ftt` — ascenso por gradiente con reinicios

- 1: **índices** $\leftarrow$ posiciones de las ocho columnas de parámetros dentro del vector

¹⁸**Optuna:** biblioteca de optimización de caja negra. Su algoritmo por defecto, TPE (*Tree-structured Parzen Estimator*), construye a partir de los intentos ya evaluados dos distribuciones de probabilidad —una de los intentos buenos y otra de los malos— y propone el siguiente punto donde el cociente entre ambas es más favorable. En la práctica: concentra los intentos en las zonas prometedoras en vez de repartirlos al azar.

¹⁹**Ascenso por gradiente:** el gradiente de una función indica la dirección de máximo crecimiento. Moverse repetidamente en esa dirección lleva a un máximo local. Es el mismo mecanismo del entrenamiento —que *desciende* por el gradiente para minimizar el error— pero aplicado a las **entradas** y con los pesos de la red congelados.

```

2: base  $\leftarrow$  contexto con esas ocho posiciones puestas a cero, normalizado con los estadísticos guardados
3:  $P \leftarrow$  matriz de proyección constante;  $lo, hi \leftarrow$  rangos de cada parámetro, normalizados
4: mejor  $\leftarrow$  ninguno; mejor_valor  $\leftarrow -\infty$ 
5: para reinicio = 1 ... X5_ASCENT_RESTARTS hacer
6:    $p \leftarrow$  punto aleatorio uniforme dentro de  $[lo, hi]$ 
7:   para paso = 1 ... X5_ASCENT_STEPS hacer
8:      $x \leftarrow \mathbf{base} + P^\top p$ 
9:     retropropagar  $-\hat{y}_{ret}(x)$  y dar un paso del optimizador Adam
10:    recortar  $p$  al rango  $[lo, hi]$  // la búsqueda no puede salirse de lo permitido
11:   fin para
12:   evaluar  $\hat{y}_{ret}$  en el  $p$  final; si supera a mejor_valor, guardarlo
13: fin para
14: desnormalizar mejor, recortar de nuevo a los rangos, redondear y convertir enteros
15: completar con los valores de config.py cualquier parámetro que no estuviera en el vector

```

Los diez reinicios desde puntos aleatorios distintos son la defensa contra los máximos locales: la superficie predicha por una red no es cóncava y un solo ascenso puede quedarse atrapado en una loma cualquiera. El recorte al rango dentro del bucle —no solo al final— importa: sin él, el ascenso puede alejarse mucho hacia una zona sin datos de entrenamiento, donde la predicción del modelo no significa nada, y volver al borde al final daría un resultado peor que haberse quedado dentro todo el camino.

3.6.4 El airbag: una regla que el modelo no puede sobrescribir

Antes de entregar la recomendación en vivo se aplica una regla explícita, no aprendida:

Pseudocódigo 3.8. _aplicar_airbag(activo, params)

```

1: leer los últimos 5 cierres horarios del activo
2: caida  $\leftarrow (p_{final} - p_{inicial}) / p_{inicial}$ 
3: si caida < -X5_AIRBAG_THRESHOLD[activo] entonces // 8 % en crypto, 5 % en acciones
4:   LOTAJES_M  $\leftarrow$  1 // lote mínimo
5:   n_sizes_ejecucion  $\leftarrow$  X5_N_MINIMO[activo] // 30 en crypto, 20 en acciones
6: fin si
7: retornar parámetros

```

La justificación es de gestión de riesgo, no de estadística: durante un desplome, los datos de entrenamiento están fuera de su distribución habitual y la recomendación del modelo no es confiable; ante la duda, se reduce exposición. Es una capa de seguridad deliberadamente tonta y deliberadamente inapelable.

El airbag **no** se aplica durante la recolección (inferir_con_contexto lo omite explícitamente). El motivo es coherente con lo anterior: en recolección se busca observar qué pasa realmente en regímenes de caída, incluso con parámetros agresivos. Aplicar el airbag ahí recortaría precisamente los datos que hacen falta para aprender de esas situaciones.

3.6.5 La salida: active_parameters.json

```

{
  "generated_at": "2026-08-26T09:12:44",
  "BTCUSD": { "model_status": "lgbm", "n_sizes_ejecucion": 173, "K": 1.2841,
              "N_EXP": 1.9022, "LAMBDA": 0.004112, "A": 8.44, "B": 3.91,
              "LOTAJES_M": 1, "PERDIDA_MAX": 187.4 },
  "TSLA": { "model_status": "untrained", "n_sizes_ejecucion": 250, ... }
}

```

La escritura es un **mezclado**, no un reemplazo: se lee el archivo existente, se actualizan solo las claves de los activos recién inferidos y se conservan las demás. Sin eso, seis procesos paralelos ejecutando `--infer --activo X` se borrarían el trabajo unos a otros. El campo `model_status` es el interruptor de seguridad que lee `X1`: con valor `untrained`, ese activo usa los valores de `config.py`.

3.7 Pseudocódigo consolidado de X5

Pseudocódigo 3.9. `main()` — los tres modos

```

1: si --status entonces mostrar recuento por activo, modelo correspondiente y contenido del
   archivo de parámetros; retornar
2: fin si
3: si --recolectar entonces
4:   si --demo entonces preguntar el activo y correr el recorrido guiado sobre el espacio de
     nombres _demo
5:   si no si el usuario elige «todos» entonces recolectar_paralelo()
6:   si no preguntar el activo y correr el recorrido guiado oficial (sin pausas)
7:   fin si
8:   retornar
9: fin si
10:                                     // a partir de aquí, --infer
11: activos  $\leftarrow$  el indicado con --activo, o todos
12: si --train entonces
13:   para cada activo hacer entrenar(activo)
14:   fin para
15: fin si
16: para cada activo hacer
17:   (tipo, modelo)  $\leftarrow$  cargar_modelo(activo)
18:   si tipo = untrained entonces
19:     parámetros  $\leftarrow$  valores de config.py; estado  $\leftarrow$  untrained
20:   si no
21:     ctx  $\leftarrow$  contexto de mercado actual
22:     parámetros  $\leftarrow$  búsqueda bayesiana o ascenso por gradiente, según tipo
23:     parámetros  $\leftarrow$  airbag(activo, parámetros)
24:   fin si
25: fin para
26: escribir config/active_parameters.json mezclando con lo ya existente

```

3.8 Parámetros de X5 y su efecto

Parámetro	Valor	Qué controla y qué pasa al subirlo
X5_MIN_TRADES_TRAIN	500	Umbral para entrenar. Bajarlo da modelos antes, pero muy ruidosos y con recomendaciones erráticas.
X5_MIN_TRADES_FTT	5 000	Umbral de cambio a red neuronal. Bajarlo arriesga sobreajuste; subirlo posterga un modelo más expresivo.
EXPLORATION_RATE	0,30	Fracción de tramos con parámetros al azar. Subirlo da más variedad y peor rendimiento simulado; bajarlo concentra los datos y limita el aprendizaje.
N_CICLOS_BT	10	Pasadas de X4 por activo y ejecución de <code>--recolectar</code> .
X5_FREQ_REGISTRO_PERIODICO	4	Velas entre filas periódicas. Bajarlo aumenta mucho el tamaño del store con filas muy correlacionadas entre sí.
X5_LAMBDA_DECAY	0,001	Olvido del pasado. Subirlo hace al modelo más reactivo y más olvidadizo.
X5_WINDOW_TRAIN	None	Tope de filas de entrenamiento. None = usar todo el store.
X5_N_OPTUNA_TRIALS	200	Intentos por inferencia con LightGBM. Subirlo mejora la búsqueda y alarga la inferencia (10–30 s con 200).
X5_ASCENT_RESTARTS	10	Puntos de partida del ascenso por gradiente. Subirlo reduce el riesgo de máximos locales, linealmente más lento.
X5_ASCENT_STEPS	300	Pasos por reinicio.
X5_ASCENT_LR	0,01	Tamaño del paso. Subirlo explora más rápido y puede oscilar sin converger.
X5_AIRBAG_THRESHOLD	0,08 / 0,05	Caída en 4 velas que dispara el airbag. Bajarlo lo hace más sensible.

Tabla 12: Parámetros de X5. Los tres primeros gobiernan el ritmo del bucle completo.

4 Interacción X4 ↔ X5

4.1 Quién llama a quién

La dependencia es mutua y vale la pena verla explícita, porque a primera vista parece circular sin serlo:

Desde	Hacia	Mecanismo
X5	X4	Subproceso: <code>python X4_backtester.py --x5 --activo {A}</code> . Comunicación por archivos y por salida estándar.
X4	X5	Importación directa: <code>import X5_macro_brain</code> , para cargar el modelo del activo e inferir parámetros.
X4	X0	Importación directa de <code>_procesar_valor_N</code> y <code>_bt_warm_start</code> .
X4	X3	Importación de <code>compute_snapshot</code> para el fotograma de indicadores técnicos.
X4	X2	Lectura del archivo <code>x2_history.json</code> , sin ejecutar nada.

Tabla 13: Llamadas entre módulos. Solo la primera cruza una frontera de proceso.

No hay ciclo de importación porque X4 importa X5 *dentro* de las funciones que lo necesitan, no en la cabecera del archivo. Y X5 nunca importa X4: lo lanza como programa externo.

4.2 Los tres relojes

Buena parte de la confusión al leer el código viene de que hay tres escalas de tiempo distintas funcionando a la vez, y las tres se llaman «ciclo» en algún comentario. Conviene separarlas de una vez:

De estos tres relojes se deduce el orden de magnitud del conjunto de datos. Con datos desde enero de 2024, un ciclo cubre unas 15 000 velas horarias por activo, es decir alrededor de 125 tramos, cada uno con su propia combinación de parámetros. Diez ciclos por activo dan unos 1 250

Reloj	Unidad	Duración	Qué cambia en cada tic
Vela	1 hora simulada	—	Se ejecutan los pasos A–F; se registra el patrimonio.
Tramo	<code>delta_recalculo_soportes</code> días (5)	≈120 velas	Se eligen parámetros nuevos y se recalculan los soportes.
Ciclo	Una pasada completa de X4	Todo el histórico	Se reentrena el modelo del activo (entre ciclos, nunca dentro).

Tabla 14: Las tres escalas temporales del pipeline X4 + X5.

experimentos distintos, cada uno generando tantas filas como operaciones cierre. Ese es el orden de magnitud que hay que alcanzar para cruzar los umbrales de 500 y 5 000 operaciones cerradas.

4.3 Una recolección completa, paso a paso

Pseudocódigo 4.1. Recorrido completo de X5 `--recolectar` para un activo

- 1: [X5] pregunta el alcance y, si procede, si reiniciar el activo
- 2: [X5] lanza el worker del activo; empieza el **ciclo 1**
- 3: [X5→X4] subproceso X4 `--x5 --activo BTCUSD`
- 4: [X4] carga velas H1 y M1; decide si empieza pasada nueva o retoma checkpoint
- 5: [X4→X5] `cargar_modelo_para_activo(BTCUSD)` → `untrained` en el primer ciclo
- 6: [X4] sin modelo, **todos** los tramos serán de exploración
- 7: [X4] arranque en frío: sortea parámetros y calcula los soportes hasta `fecha_inicio`
- 8: [X4] avanza vela a vela ejecutando los pasos A–F
- 9: [X4] cada operación cerrada → una fila 'oc' en `BTCUSD_store.csv`
- 10: [X4] cada 4 velas con posiciones abiertas → una fila 'periodico'
- 11: [X4] cada 5 días simulados → parámetros nuevos y soportes recalculados
- 12: [X4] llega al final de los datos (o quiebra); guarda checkpoint y termina
- 13: [X5] cuenta las filas 'oc' del store
- 14: **si** ≥ 500 **entonces** [X5] entrena LightGBM y guarda modelo, columnas y métricas
- 15: **fin si**
- 16: [X5] empieza el **ciclo 2**
- 17: [X4→X5] ahora sí hay modelo: 70 % de los tramos usarán inferencia
- 18: [X4] en cada tramo de explotación construye el contexto *al instante simulado* y pide parámetros a X5
- 19: [X4] las filas nuevas del store ya reflejan decisiones informadas por el modelo
- 20: ... hasta agotar `N_CICLOS_BT` o superar 5 000 operaciones cerradas

4.4 Aislamiento: qué separa a qué

Tres ejes de aislamiento operan simultáneamente y es fácil confundirlos:

1. **Por activo.** `--activo BTCUSD` redirige las carpetas de trabajo de X4 a `bt_BTCUSD`. Permite seis procesos simultáneos sin colisión. El store ya era por activo por diseño.
2. **Por espacio de nombres.** La variable de entorno `X5_DEMO_SUFFIX` añade `_demo` al store, al directorio del modelo, al historial de métricas y a las carpetas de backtesting. Es lo que permite que el modo demostración no toque nada real.
3. **Por versión.** Las versiones V0, V1, ... de X4 tienen cada una su configuración congelada y su propia carpeta de resultados. Este eje es independiente de los otros dos: el pipeline de X5 no usa versiones.

4.5 Supuestos y riesgos conocidos

Vale la pena dejar por escrito lo que puede salir mal, porque nada de esto es un defecto que se pueda «arreglar» sin más: son propiedades del enfoque.

El modelo aprende de datos que él mismo generó.

A partir del segundo ciclo, el 70 % de los tramos usa parámetros recomendados por el modelo. Si el modelo se equivoca de forma sistemática, generará datos que confirmen su error. La exploración permanente del 30 % es el contrapeso, pero es un contrapeso, no una garantía.

El backtest no es la realidad.

Todo lo que el modelo aprende viene de una simulación con supuestos optimistas: sin comisiones, sin deslizamiento, con intra-vela sintética. Los parámetros óptimos *para el simulador* pueden no serlo para el mercado. La Fase 2 —realimentar con operaciones reales— existe precisamente para corregir eso.

Ocho parámetros y un objetivo ruidoso.

El retorno de una operación individual tiene una componente de azar enorme. El modelo puede necesitar muchísimos datos para separar la señal del ruido, y un R^2 negativo en las primeras iteraciones es lo esperable, no una señal de fallo.

Contexto de cartera vacío en la inferencia en vivo.

En Fase 1, las columnas de cartera se rellenan con ceros al inferir en vivo, mientras que en el store esas mismas columnas traen valores reales. Es un desajuste entre entrenamiento e inferencia que solo se resuelve cuando X1 provea el estado real de la cartera.

Las columnas OA se aproximan con las de OE.

Al inferir en vivo se copian los valores del momento de crear la orden como si fueran los del momento de ejecutarla. Es una aproximación razonable para órdenes que se ejecutan pronto y mala para las que tardan semanas.

4.6 Cómo saber si el bucle está funcionando

Tres señales, en orden de disponibilidad:

1. `python scripts/X5_macro_brain.py --status` muestra el recuento de operaciones cerradas por activo y qué modelo corresponde. Si no crece entre recolecciones, X4 no está cerrando posiciones y hay que mirar los parámetros o los datos antes que el modelo.
2. El historial en `resources/x5/Performance/{ACTIVO}_performance.json` muestra la evolución de las métricas de validación entrenamiento tras entrenamiento. La señal buena es que el error de validación baje mientras el store crece.
3. `equity_global.csv` de las carpetas `bt_{ACTIVO}` da la curva de capital de la última pasada. Sirve para juzgar si las configuraciones que el modelo recomienda producen pasadas mejores que las de exploración pura.

5 Anexos

5.1 Glosario técnico

Término	Definición
Arranque en caliente	Iniciar un algoritmo iterativo desde una solución previa conocida en vez de un punto aleatorio.
Ascenso por gradiente	Optimizar moviéndose repetidamente en la dirección de máximo crecimiento de una función derivable.
Atención	Mecanismo por el que cada elemento de una secuencia pondera cuánto atender a los demás. Base del transformer.
Backtesting	Evaluar una estrategia sobre datos históricos, simulando que se opera en cada momento del pasado.
Escritura atómica	Escribir a un archivo temporal y renombrarlo sobre el destino, de modo que nunca quede a medio escribir.
FT-Transformer	Arquitectura transformer adaptada a datos tabulares: cada columna se convierte en un vector antes de aplicar atención.
Gradient boosting	Método que construye árboles de decisión en secuencia, cada uno corrigiendo el error de los anteriores.
Hueco (<i>gap</i>)	Salto de precio entre el cierre de una vela y la apertura de la siguiente, sin cotizaciones intermedias.
LightGBM	Implementación eficiente de gradient boosting; estándar en datos tabulares.
Margen	Garantía inmovilizada al abrir una posición apalancada: valor de la posición dividido por el apalancamiento.
Nivel de margen	Patrimonio dividido por margen usado, en porcentaje. Indicador de supervivencia de la cuenta.
Modelo sustituto	Modelo barato que imita una función cara de evaluar, para poder optimizarla.
Normalización de capa	Reescalado a media cero y desviación uno dentro de cada vector; estabiliza redes profundas.
Optuna / TPE	Biblioteca de optimización de caja negra que concentra los intentos en las zonas prometedoras del espacio.
Orden límite de compra	Instrucción de comprar automáticamente si el precio baja hasta un nivel dado.
Parada temprana	Detener el entrenamiento cuando el error de validación deja de mejorar, para evitar sobreajuste.
Patrimonio (<i>equity</i>) R^2	Saldo realizado más el resultado flotante de las posiciones abiertas. Fracción de la variabilidad del objetivo explicada por el modelo. Negativo significa peor que predecir la media.
Sobreajuste	Memorizar el ruido del conjunto de entrenamiento, a costa del rendimiento con datos nuevos.
Trailing stop	Stop de venta cuyo nivel sube con el precio y nunca baja.
Vela (OHLCV)	Resumen de un intervalo de tiempo: apertura, máximo, mínimo, cierre y volumen.

5.2 Mapa de archivos generados

Ruta	Contenido
resources/x4/version{V}/	Configuración congelada y resultados de una versión de backtesting.
.../resources_{V}/checkpoint.json	Estado completo para reanudar.
.../resources_{V}/trades.json	Una entrada por operación cerrada.
.../resources_{V}/events.json	Bitácora granular de creaciones, cancelaciones, stops y cierres.
.../resources_{V}/equity_global.csv	Saldo, patrimonio y margen por vela.
.../resources_{V}/equity_activos.csv	Ganancia cerrada, flotante y total por vela y activo.
.../conjuntos_N/{A}_{N}_bt.json	Caché de soportes indexado por fecha.
.../conjuntos_N/{A}_{N}_bt_delta.json	Paso adaptativo del optimizador de soportes.
resources/x5/{A}_store.csv	Conjunto de datos de entrenamiento del activo.
resources/x5/models/{A}/	Modelos entrenados y orden de columnas.
resources/x5/Performance/{A}_performance.json	Historial de métricas por entrenamiento.
resources/x5/bt_{A}/	Carpeta de backtesting aislada del activo (modo X5).
resources/x5/demo_plots/{A}/	Gráficos de cada búsqueda de soportes del recorrido guiado.
config/active_parameters.json	Recomendación vigente por activo, con su estado de modelo.

5.3 Índice de funciones

Referencia rápida para saltar del documento al código. Los números de línea corresponden al estado del repositorio en la fecha de este documento y son orientativos.

5.3.1 X4_backtester.py

Función	Línea	Propósito
_cargar_config	36	Carga la configuración congelada de una versión.
_cargar_config_x5	47	Carga config_x5_default.py para el modo X5.
_cargar_datos_h1	68	Lee, ordena y recorta las velas horarias por activo.
_cargar_datos_m1	89	Ídem para velas de un minuto; admite ausencia.
_cargar_checkpoint	107	Restaura el estado, reconvirtiendo claves de texto a número.
_guardar_checkpoint	120	Vuelca el estado a JSON.
_worker_recalcular	144	Adaptador que invoca X0._procesar_valor_N en un proceso aparte.
_recalcular_soportes	228	Orquesta el recálculo paralelo y recarga los soportes del caché.
_calcular_estado_cuenta	267	Saldo, patrimonio, margen usado y libre, nivel de margen.
_id_orden	310	Identificador reconstruible de una operación.
_registrar_trade	320	Arma la entrada de trades.json.
_paso_A	361	Cancela órdenes en espera sin soporte vigente.
_paso_B	380	Ejecuta órdenes alcanzadas; trata huecos y control de margen.
_paso_C	465	Activa y arrastra el stop protector.
_paso_D	495	Cierra por pérdida máxima.
_paso_E	533	Cierra por stop tocado.

Función	Línea	Propósito
<code>_paso_F</code>	570	Crea órdenes en espera sobre soportes libres.
<code>_necesita_intravela</code>	611	Decide si la vela requiere resolución minuto a minuto.
<code>_escalar_bloque_m1</code>	629	Deforma un tramo M1 real para encajarlo en la vela horaria.
<code>_simular_intravela</code>	655	Repite los pasos B–F sobre 60 velas de un minuto.
<code>_procesar_candle</code>	675	Aplica los pasos A–F a una vela de un activo.
<code>_append_equity</code>	708	Escribe las dos curvas de patrimonio.
<code>_flush_json_list</code>	745	Volcado atómico con reintentos de operaciones y eventos.
<code>_features_temporales_ts</code>	777	Features de calendario, incluidos festivos bursátiles.
<code>_compute_x5_snapshot</code>	828	Fotograma de indicadores X3 y score X2 al instante simulado.
<code>_contexto_portfolio_x5</code>	840	Estado de la cartera del activo como features.
<code>_construir_fila_oc</code>	862	Convierte una operación cerrada en fila de entrenamiento.
<code>_construir_fila_periodica</code>	902	Fila de seguimiento con posiciones abiertas.
<code>_append_x5_store</code>	932	Añade una fila al store, con defensa contra archivos truncados.
<code>ejecutar_backtest</code>	956	Bucle principal de la simulación.
<code>_params_explore_activo</code>	1249	Sorteo de parámetros dentro de los rangos.
<code>_contexto_inferencia_x5</code>	1273	Contexto al instante simulado para pedir inferencia.
<code>_seleccionar_params_x5</code>	1299	Decide explorar o explotar por activo.
<code>_aplicar_params_x5</code>	1352	Injecta los parámetros del tramo en la configuración viva.
<code>ejecutar_x5_ciclo</code>	1372	Prepara el aislamiento por activo y carga el modelo del ciclo.

5.3.2 X5_macro_brain.py

Función	Línea	Propósito
<code>_cargar_store</code>	128	Lee el CSV del activo descartando filas malformadas.
<code>_n_oc</code>	138	Cuenta filas de tipo operación cerrada.
<code>_seleccionar_tipo</code>	144	Elige <code>untrained</code> , <code>lgbm</code> o <code>ftt</code> según el volumen.
<code>_params_baseline</code>	153	Valores de <code>config.py</code> , usados como reserva.
<code>_feature_cols_de_store</code>	168	Fija el orden canónico de columnas y excluye las de cierre.
<code>_preparar_features</code>	192	Construye X , y y los pesos temporales.
<code>_leer_contexto_actual</code>	302	Contexto de mercado en vivo.
<code>_construir_vector</code>	336	Combina contexto fijo y parámetros variables en el orden correcto.
<code>_calcular_metricas</code>	346	MAE, MSE, MAPE y R^2 .
<code>_guardar_performance</code>	365	Añade una entrada al historial de métricas.
<code>_entrenar_lgbm</code>	396	Entrena los tres modelos de árboles con división temporal.
<code>FTTransformer</code>	520	Red con tronco compartido y tres cabezas de salida.
<code>_entrenar_ftt</code>	558	Entrena la red, normalizando con estadísticos del tramo de entrenamiento.
<code>_aplicar_airbag</code>	679	Fuerza exposición mínima tras una caída brusca.
<code>_inferir_lgbm</code>	718	Búsqueda bayesiana de parámetros con Optuna.

Función	Línea	Propósito
<code>_inferir_ftt</code>	771	Ascenso por gradiente sobre las entradas, con reinicios.
<code>_entrenar</code>	876	Elige y entrena el modelo que corresponda.
<code>cargar_modelo_para_activo</code>	893	Carga el modelo una sola vez, para reuso desde X4.
<code>inferir_con_contexto</code>	910	Inferencia con contexto inyectado, sin airbag.
<code>_escribir_active_parameters</code>	948	Mezcla y escritura atómica del archivo de parámetros.
<code>_mostrar_status</code>	976	Diagnóstico por activo.
<code>_worker_recolectar_activo</code>	1018	Bucle de ciclos y reentrenamiento de un activo.
<code>_recolectar</code>	1090	Orquestador paralelo de la recolección.
<code>_borrar_checkpoints_activo</code>	1196	Borrado completo del activo en su espacio de nombres.
<code>_recolectar_guiado</code>	1248	Recorrido secuencial narrado, en modo demostración u oficial.
<code>main</code>	1365	Despacho de los tres modos de la línea de comandos.

Documento generado a partir del código en `scripts/X4_backtester.py`, `scripts/X5_macro_brain.py`, `scripts/config_x5_default.py` y `scripts/config.py`.

Fuente L^AT_EX: `docs/plans/algoritmos.tex`. Compilar con `tectonic algoritmos.tex`.